Szakdolgozat



Miskolci Egyetem

Webalapú SVG-szerkesztő alkalmazás

Készítette:

Czibik Lóránt Patrik

Programtervező informatikus

Témavezető:

Dr. Vadon Viktória

Miskolc, 2026

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

Szakdolgozat Feladat

Czibik Lóránt Patrik (K1IFAB) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: Webfejlesztés, vektorgrafika, SVG

A szakdolgozat címe: Webalapú SVG szerkesztő alkalmazás

A feladat részletezése:

Az SVG (Scalable Vector Graphics) egy XML formátumú vektorgrafikus leíró nyelv, amely széles körben használatos illusztrációk, diagramok és ikonok készítésére. A formátum jellegzetessége, hogy a grafikus elemeket matematikai leírással tárolja, így az ábrák korlátlanul nagyíthatók minőségromlás nélkül.

A dolgozat célja egy olyan webalapú szerkesztőfelület tervezése és elkészítése, amely lehetővé teszi az SVG ábrák létrehozását és szerkesztését grafikus eszközökkel, a kimenet pedig szerkeszthető SVG fájlként vagy képként menthető el.

A dolgozat tartalmazza a hasonló célú alkalmazások funkcióinak áttekintését és összehasonlítását, a szerkesztőfelület specifikációját, valamint annak vizsgálatát, hogy a bemeneti SVG fájlok milyen esetekben és milyen korlátozásokkal módosíthatók a szerkesztőfelületen.

Témavezető: Dr. Vadon Viktória, egyetemi adjunktus

A feladat kiadásának ideje: 2025. 09. 18.

.....
szakfelelős

Eredetiségi Nyilatkozat

Alulírott **Czibik Lóránt Patrik**; Neptun-kód: `K1IFAB` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *Webalapú SVG szerkesztő alkalmazás* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ... év ..... hó ..... nap

.....
Hallgató

1. A szakdolgozat feladat módosítása
   - szükséges (módosítás külön lapon)
   - nem szükséges

....
dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás):	konzulens (dátum, aláírás):
.....	.....
.....	.....
.....	.....

3. A szakdolgozat beadható:

....
dátum témavezető(k)

4. A szakdolgozat
   - ..... szövegoldalt
   - ..... program protokollt (listát, felhasználói leírást)
   - ..... elektronikus adathordozót (részletezve)
   - .....
   - ..... egyéb mellékletet (részletezve)
   - .....

tartalmaz.

....
dátum témavezető(k)

5. A szakdolgozat bírálatra
   - bocsátható
   - nem bocsátható

A bíráló neve: .....

....
dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata:	.....
a bíráló javaslata:	.....
a szakdolgozat végleges eredménye:	.....

Miskolc, ...
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés **1**

2. Az SVG és eszközkészlete **2**
2.1. Az SVG grafikus elemei . . . 2
2.1.1. Megjelenési attribútumok . . . 3
2.1.2. Alapalakzatok . . . 5
2.1.3. Útvonalak . . . 6
2.1.4. Szöveg és kép . . . 8
2.1.5. Csoportok, definíciók és színátmenetek . . . 9
2.2. Szerkesztőeszközök . . . 11
2.2.1. Inkscape . . . 12
2.2.2. SVG-Edit . . . 13
2.2.3. Boxy SVG . . . 14
2.2.4. Figma . . . 15
2.2.5. Mediamodifier . . . 16
2.2.6. Összehasonlítás . . . 16
2.2.7. Következtetések és célkitűzések . . . 17

3. Tervezés **18**
3.1. Követelmények . . . 18
3.1.1. Funkcionális követelmények . . . 18
3.1.2. Nem funkcionális követelmények . . . 18
3.2. A felhasználói felület terve . . . 20
3.3. Technológiai választások . . . 21
3.3.1. Svelte és TypeScript . . . 21
3.3.2. Fejlesztői eszközök . . . 21
3.3.3. Geometriai és betűtípus-könyvtárak . . . 22
3.4. Az alkalmazás rétegei . . . 23
3.5. Adatmodell . . . 24
3.6. Az előzmények kezelése . . . 25
3.7. Az eszközök közös felülete . . . 26
3.8. Tárolás és fájlformátumok . . . 27

4. Megvalósítás **28**
4.1. A fejlesztés menete . . . 28
4.2. A projekt felépítése . . . 28
4.3. Megjelenítés és találatvizsgálat . . . 29
4.4. Nézet, nagyítás és illesztés . . . 29
4.5. Az előzmények megvalósítása . . . 30

4.6. Rajzoló eszközök . . . 31
4.6.1. Toll és körív . . . 31
4.6.2. Szabadkézi rajz . . . 32
4.7. Alakzatok szerkesztése . . . 32
4.7.1. Elforgatott alakzatok méretezése . . . 32
4.7.2. A forgatási középpont . . . 33
4.7.3. Csomópontok szerkesztése . . . 33
4.8. Logikai műveletek és vágás . . . 34
4.9. Körvonalak . . . 35
4.9.1. Szaggatott vonalak . . . 35
4.9.2. Körvonal útvonallá alakítása és a foltecset . . . 35
4.10. Szövegek és betűtípusok . . . 36
4.10.1. Szövegszerkesztés és mérés . . . 36
4.10.2. Betűtípusok forrásai . . . 36
4.10.3. Szöveg útvonallá alakítása . . . 37
4.11. SVG-importálás és -exportálás . . . 37
4.11.1. Importálás . . . 37
4.11.2. Exportálás . . . 38
4.12. Vágólap és tárolás . . . 39
4.13. Reszponzív felület és érintés . . . 39

5. Tesztelés és használat **41**
5.1. Tesztelési stratégia . . . 41
5.2. Automatikus tesztek . . . 41
5.3. Véletlenszerű tesztelés . . . 43
5.4. Betűtípusok tesztelése hálózat nélkül . . . 44
5.5. Kézi tesztelés és regressziós tesztek . . . 44
5.6. Böngészős ellenőrzés és a kész alkalmazás mérete . . . 44
5.7. Az alkalmazás használata . . . 45

6. Az eredmények értékelése **48**
6.1. A követelmények teljesülése . . . 48
6.2. Mérések . . . 49
6.2.1. Az importálás pontossága valós fájlokon . . . 49
6.2.2. A szöveg útvonallá alakításának pontossága . . . 50
6.3. Összevetés a vizsgált szerkesztőkkel . . . 51
6.4. Nem várt tapasztalatok . . . 52
6.5. Szakmai jelentőség és alkalmazhatóság . . . 52

7. Összefoglalás **53**

Summary **54**

Források **55**

1. fejezet

Bevezetés

A Scalable Vector Graphics (SVG) [73] XML-alapú leírónyelv kétdimenziós vektorgrafikák készítéséhez. Első változatát a W3C 2001-ben tette ajánlássá, hogy a webnek legyen szabványos vektorgrafikus formátuma [72]. Az SVG a képet alakzatok, szövegek és beágyazott képek matematikai leírásaként tárolja, így az minőségromlás nélkül tetszőlegesen nagyítható. A rasztergrafika [4] ezzel szemben képpontok rácsával ábrázolja a képet, és inkább fényképekhez alkalmas. Az SVG mára a web egyik alapformátuma. A W3Techs 2026. novemberi felmérése szerint a weboldalak 68,8%-a használ SVG-képet [79], jellemzően ikonokhoz, logókhoz és diagramokhoz.

Egy SVG-fájl szövegszerkesztővel is módosítható, de egy görbe vagy egy összetett alakzat így nehezen kezelhető, ezért a gyakorlatban grafikus szerkesztőt használnak hozzá. A 2. fejezetben vizsgált szerkesztők között van nagy tudású, de asztali vagy fizetős program, és van egyszerű, de korlátozott webes eszköz is. Szakdolgozatom célja egy olyan webalapú SVG-szerkesztő tervezése és megvalósítása, amely a böngészőben, telepítés és regisztráció nélkül fut, mégis teljes értékű útvonalszerkesztést kínál, és telefonon is kezelhető. Az elkészült alkalmazás a *Solid SVG* nevet kapta. A feladatkiírás külön kérdése, hogy a beolvasott SVG-fájlok milyen esetekben és milyen korlátozásokkal módosíthatók, hiszen tartalmazhatnak szkripteket vagy a szerkesztő által nem kezelt szerkezeteket.

Egy ilyen szerkesztő elsőre egyszerű feladatnak tűnhet, hiszen a böngésző maga is meg tudja jeleníteni az SVG ábrákat. A nehézségek a szerkesztésnél kezdődnek. Egy elforgatott alakzat átméretezésekor a szemközti saroknak a helyén kell maradnia, két alakzat egyesítésekor a görbéknek görbének kell maradniuk, az útvonallá alakított szövegnek pedig pontosan úgy kell kinéznie, mint a képernyőn, akkor is, ha a betűtípus egy másik gépen nincs telepítve. Mindezt szerver nélkül kell megoldani, úgy, hogy minden művelet visszavonható legyen. A megvalósításhoz a Svelte keretrendszert [64] és a TypeScript nyelvet [51] választottam.

A 2. fejezet az SVG grafikus elemeit és a hasonló célú alkalmazásokat mutatja be, és ezek alapján megfogalmazza a célkitűzéseket. A 3. fejezet a követelményeket és az alkalmazás tervét ismerteti, a 4. fejezet a megvalósítás érdekesebb részeit, az 5. fejezet a tesztelést és a használatot. A 6. fejezet a célkitűzések teljesülését értékeli, a 7. fejezet pedig magyarul és angolul is összefoglalja a munkát.

2. fejezet

Az SVG és eszközkészlete

A fejezet első fele az SVG azon elemeit mutatja be, amelyekkel egy szerkesztőnek dolgoznia kell, a második fele pedig öt elterjedt SVG-szerkesztőt hasonlít össze. Az összehasonlítás szempontjai adják a 3. fejezetben megfogalmazott követelmények vázát.

2.1. Az SVG grafikus elemei

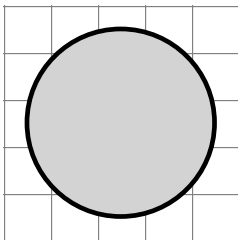
A Scalable Vector Graphics (SVG) XML-alapú nyelv kétdimenziós vektorgrafikák leírására. A dokumentum hierarchikus felépítésű, minden grafikus objektum egy közös gyökérelembe, az <svg> kezdő és záró címkéje közé kerül, ahogyan az alábbi kódrészlet mutatja.

```
<svg xmlns="http://www.w3.org/2000/svg"
    width="100" height="100">
    <!-- SVG-elemek -->
</svg>
```

Az SVG koordináta-rendszerének origója a bal felső sarokban van, és a matematikai ábrázolással ellentétben az y tengely lefelé mutat.

A `viewBox` attribútum Az <svg> elem opcionális viewBox attribútuma egy saját koordináta-rendszert határoz meg a tartalom számára, amelyet a böngésző a `width` és `height` attribútumokban megadott méretre skáláz. Az értéke négy szám: `min-x`, `min-y`, `width` és `height`. Ha hiányzik, egy koordinátaegység egy CSS-képpontnak felel meg. A 2.1. ábrán a `viewBox` egy 0-tól 100-ig terjedő koordináta-rendszert ad meg, a tartalom pedig kitölti a 400×400 képpontos területet.

```
<svg xmlns="http://www.w3.org/2000/svg"
    width="400" height="400" viewBox="0 0 100 100">
    <circle cx="50" cy="50" r="40"
        fill="lightgray" stroke="black" stroke-width="2"/>
</svg>
```



2.1. ábra. A viewBox attribútum hatása a koordináta-rendszerre

A szakasz először a megjelenést befolyásoló attribútumokat ismerteti (2.1.1. alszakasz), majd az alapalakzatokat (2.1.2. alszakasz), az útvonalakat (2.1.3. alszakasz), a

szöveget és a képeket (2.1.4. alszakasz), végül a csoportosítást, az újrafelhasználható definíciókat és a színátmeneteket (2.1.5. alszakasz). Az elemek használatát kódrészletek és a hozzájuk tartozó ábrák mutatják be.

2.1.1. Megjelenési attribútumok

Az elemek kinézetét a geometriájuk mellett a megjelenési attribútumok határozzák meg [42, 76]. Ezek CSS-tulajdonságként is megadhatók, így szerepelhetnek attribútumként az elemen, a `style` attribútumban vagy egy stíluslapon.

Körvonal

A körvonal az alakzat határvonala. Az SVG a színe és vastagsága mellett a vonal végződését, a sarkok alakját és a szaggatást is leírja.

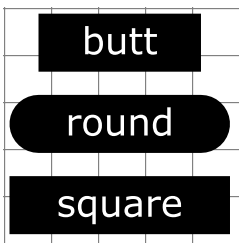
`stroke` A körvonal színe. Értéke színkód, színnév vagy hivatkozás egy színátmenetre. Alapértéke `none`, vagyis a körvonal nem látszik.

`stroke-width` A vonalvastagság az aktuális koordináta-rendszer egységében. Alapértéke 1.

`stroke-opacity` A körvonal átlátszatlansága 0 és 1 között, ahol 0 a teljesen átlátszó. A kitöltés átlátszatlanságát nem befolyásolja.

`stroke-linecap` A nyitott útvonalak végpontjainak alakja, a 2.2. ábra szerint.

- `butt` A vonal pontosan a végpontban ér véget. Ez az alapérték.
- `round` A vonal végét egy félkör zárja le, amelynek sugara a vonalvastagság fele.
- `square` A vonal négyzetes véggel a vonalvastagság felével túlnyúlik a végponton.

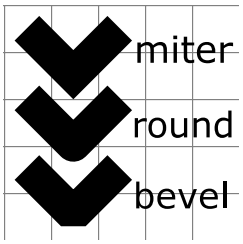


2.2. ábra. A stroke-linecap attribútum három értéke

`stroke-linejoin` Két szakasz találkozásánál a sarok alakja, a 2.3. ábra szerint.

- `miter` Hegyes sarok, a külső élek a metszéspontjukig hosszabbodnak meg. Ez az alapérték.
- `round` Lekerekített sarok.
- `bevel` Levágott sarok, a két szakasz külső sarokpontjait egy egyenes köti össze.

Az SVG 2 két további értéket vezetett be, a `miter-clip` és az `arcs` értéket, ezeket azonban a böngészők még nem támogatják széles körben [40].



2.3. ábra. A stroke-linejoin attribútum három értéke

`stroke-dasharray` A szaggatott vonal mintáját adja meg vesszővel vagy szóközzel elválasztott számokkal, amelyek felváltva a vonalszakaszok és a hézagok hosszát adják meg. A hosszak az aktuális koordináta-rendszer egységében értendők, a minta tehát nem igazodik a vonalvastagsághoz.

`stroke-dashoffset` A szaggatási minta kezdőpontjának eltolása az útvonal elejéhez képest. Vele a minta megváltoztatása nélkül csúsztatható el a szaggatás.

Kitöltés

A kitöltés az alakzat belső területét színezi. Nyitott útvonalnál az a terület számít belsőnek, amely a végpontok egyenes összekötésével jönne létre.

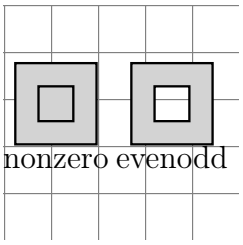
`fill` A belső terület színe. A körvonalhoz hasonlóan lehet szín, színátmenet vagy minta. Alapértéke fekete, a `none` érték mellett a belső terület üres marad.

`fill-opacity` A kitöltés átlátszatlansága 0 és 1 között. A körvonal átlátszatlanságát nem befolyásolja.

`fill-rule` Önmagát metsző vagy lyukas útvonalaknál azt dönti el, mely területek számítanak belsőnek. A vizsgált pontból tetszőleges irányban egy félegyenest húzunk, és az útvonallal alkotott metszéspontjait vizsgáljuk.

`nonzero` Az alapszabály. Ahol az útvonal az egyik irányban keresztezi a félegyenest, ott eggyel nő, ahol a másikban, ott eggyel csökken egy számláló. A pont akkor belső, ha a számláló végül nem nulla.

`evenodd` A pont akkor belső, ha a metszéspontok száma páratlan. Ezzel a szabállyal egy külső és egy belső kontúrból egyszerűen készíthető lyukas alakzat.



2.4. ábra. A fill-rule attribútum két értéke

Átlátszóság

Az `opacity` attribútum az egész elem átlátszatlanságát adja meg 0 és 1 között. A `fill-opacity` és a `stroke-opacity` attribútumtól abban különbözik, hogy csoportra alkalmazva a csoport tartalmát egy egységként halványítja el. Az egymást fedő tagok ezért nem látszanak át egymáson, csak a csoport egésze lesz áttetsző.

Transzformációk

A `transform` attribútum az elem koordináta-rendszerét alakítja át [50]. Az elemet a `translate(x, y)` függvény eltolja, a `rotate(a, x, y)` az (x, y) pont körül a fokkal elforgatja, a `scale(x, y)` pedig a két tengely mentén méretezi. Egyetlen paraméterrel, `scale(s)` alakban mindkét irányban ugyanakkora a méretezés. A `skewX` és `skewY` függvény nyírást ír le, a `matrix` pedig hat számmal tetszőleges affin transzformációt, vagyis az eltolás, a forgatás, a méretezés és a nyírás bármely kombinációját. Több transzformáció egymás után is felsorolható.

Szűrők, vágás, maszkolás és animáció

Az SVG ennél több megjelenítési lehetőséget is kínál, ezek azonban túlmutatnak az elkészült szerkesztőn. A `filter` elem szűrőprimitívekből épít fel összetett hatásokat, például elmosást vagy árnyékot [46]. A `clipPath` elem vágóútvonalat határoz meg, amelyen kívül az elem nem látszik [44], a `mask` elem pedig a maszk világosságával vagy átlátszatlanságával fokozatosan teszi láthatóvá az elemet [48]. Az attribútumok időbeli változása a SMIL nyelvből átvett `animate`, `animateTransform` és `animateMotion` elemekkel írható le [41]. Ezek kezelése a továbbfejlesztési lehetőségek közé tartozik, a szerkesztő importálója figyelmeztetést ad, ha egy fájl ilyen elemeket tartalmaz.

2.1.2. Alapalakzatok

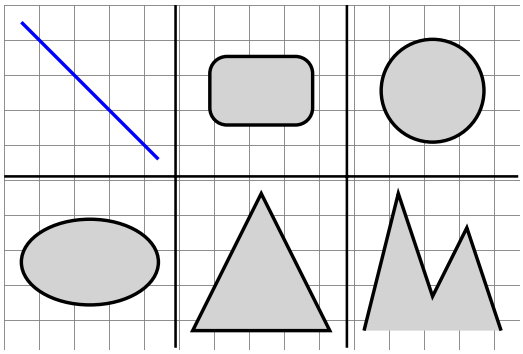
Az SVG hat alapalakzatot ismer [43], amelyek egy-egy egyszerű geometriai formát néhány attribútummal írnak le. A 2.1. táblázat az alakzatokat meghatározó attribútumokat foglalja össze, a 2.5. ábra pedig mind a hatot bemutatja.

2.1. táblázat. Az SVG alapalakzatai és meghatározó attribútumaik

Alakzat	**Elem**	**Attribútumok**
vonal	`<line>`	`x1`, `y1` és `x2`, `y2`: a két végpont
téglalap	`<rect>`	`x`, `y`: a bal felső sarok; `width`, `height`: a méret; `rx`, `ry`: a sarkok lekerekítésének sugara
kör	`<circle>`	`cx`, `cy`: a középpont; `r`: a sugár
ellipszis	`<ellipse>`	`cx`, `cy`: a középpont; `rx`, `ry`: a vízszintes és a függőleges sugár
sokszög	`<polygon>`	`points`: a csúcsok `x,y` párjai bejárási sorrendben
töröttvonal	`<polyline>`	`points`: a töréspontok, mint a sokszögnél

A sokszög utolsó csúcsát az elem automatikusan összeköti az elsővel, ezért a kezdőpontot nem kell megismételni. A töröttvonal ugyanígy írható le, de nyitva marad. A kitöltés ennek ellenére a zárt területre vonatkozik, ahogyan a 2.5. ábra jobb alsó részén a szürke terület mutatja.

```
<line x1="10" y1="10" x2="90" y2="90"
  stroke="blue" stroke-width="2"/>
<rect x="20" y="30" width="60" height="40"
  rx="10" ry="10"/>
<circle cx="50" cy="50" r="30"/>
<ellipse cx="50" cy="50" rx="40" ry="25"/>
<polygon points="50,10 90,90 10,90"/>
<polyline
  points="10 90, 30 10, 50 70, 70 30, 90 90"/>
```



2.5. ábra. A hat alapalakzat. Mindegyik a saját 100 × 100 egységes cellájában áll, a kitöltés világosszürke, a körvonal fekete.

2.1.3. Útvonalak

A <path> elem a legáltalánosabb alakzat, minden más alakzat leírható vele. Az alakját a d attribútum parancsai adják meg. Minden parancs egy betűből és a paramétereiből áll. A nagybetűs parancsok paraméterei abszolút koordináták, a kisbetűseké az aktuális ponthoz képesti eltolások. A paraméterek többnyire koordinátapárok, az ív azonban szöget és két jelzőbitet is kap.

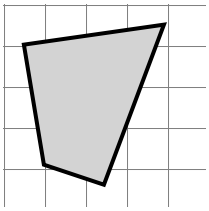
Egyenes szakaszok

Az egyenes szakaszokat az alábbi parancsok írják le.

- M, m: áthelyezi az aktuális pontot rajzolás nélkül, mintha a tollat felemelnénk
- L, l: egyenes szakasz az aktuális ponttól a megadott pontig
- H, h: vízszintes szakasz
- V, v: függőleges szakasz
- Z, z: egyenes szakasszal visszazárja az útvonalat a kezdőpontjába

A 2.6. ábra útvonala egy négyszöget rajzol, amely a $(10, 20)$ pontból indul, és a Z paranccsal zárul.

```
<path d="M 10,20 L 80,10 L 50,90 L 20,80 Z"
   fill="lightgray" stroke="black"
   stroke-width="2"/>
```



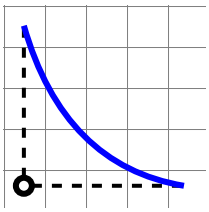
2.6. ábra. Egyenes szakaszokból álló zárt útvonal kódja és megjelenése

Bézier-görbék

A Bézier-görbék alakját kontrollpontok határozzák meg [34, 56, 10]. A görbe a két végpontján áthalad, a kontrollpontokon általában nem, azok csak a görbe irányát és ívét szabják meg. A grafikus szerkesztők a kontrollpontokat fogantyúként jelenítik meg, amelyek a végpontokhoz kötött szakaszok végén mozgathatók. Az SVG a másod- és a harmadfokú Bézier-görbét ismeri.

Másodfokú Bézier-görbe. Egyetlen kontrollpontja van, amely a kezdő- és a végpont érintőjének irányát egyszerre határozza meg. A `Q`, `q` paranccsal rajzolható.

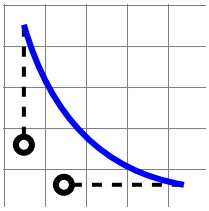
```
<path d="M 90,90 Q 30,80 10,10"
    fill="none" stroke="blue"
    stroke-width="3"/>
```



2.7. ábra. Másodfokú Bézier-görbe a kontrollpontjával

Harmadfokú Bézier-görbe. Két kontrollpontja közül az első a görbe kezdeténél, a második a végénél határozza meg az irányt. A `C`, `c` paranccsal rajzolható.

```
<path d="M 90,90 C 50,83 23,57 10,10"
    fill="none" stroke="blue"
    stroke-width="3"/>
```



2.8. ábra. Harmadfokú Bézier-görbe a két kontrollpontjával

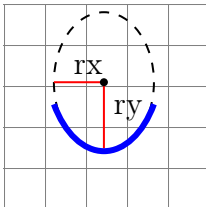
Ív

Az `A`, `a` parancs egy ellipszis ívét rajzolja az aktuális ponttól a megadott végpontig. Ez a legtöbb paraméterrel rendelkező útvonalparancs:

- `rx`, `ry`: az ellipszis vízszintes és függőleges sugara
- `x-axis-rotation`: az ellipszis x tengelyének elforgatási szöge
- `large-arc-flag`: a kezdő- és végponton átmenő lehetséges ívek közül a legfeljebb 180 fokosat (0) vagy a nagyobbat (1) választja
- `sweep-flag`: az ív haladási iránya, 1 esetén pozitív, 0 esetén negatív szögirányban; mivel az y tengely lefelé mutat, a pozitív irány a képernyőn az óramutató járásával egyezik meg
- `x`, `y`: az ív végpontja

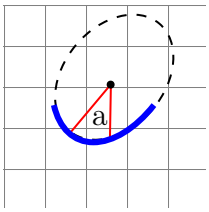
A következő négy ábra a paraméterek hatását mutatja. A kód pirossal kiemelt része az előző ábrához képest változott.

```
<path d="M 25,50
    A 29.5,50 0 0 0 75,50"
    fill="none" stroke="blue" stroke-width="3">
</path>
```



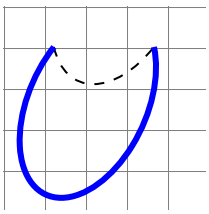
2.9. ábra. Az rx és ry paraméter hatása. A kék ív egy képzeletbeli ellipszis része, a piros vonalak a sugarakat jelölik.

```
<path d="M 25,50
    A 29.5,50 25 0 0 75,50"
    fill="none" stroke="blue" stroke-width="3">
</path>
```



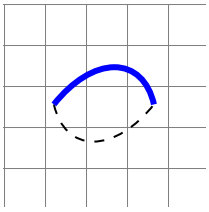
2.10. ábra. Az x-axis-rotation paraméter hatása

```
<path d="M 25,50
    A 29.5,50 25 1 0 75,50"
    fill="none" stroke="blue" stroke-width="3">
</path>
```



2.11. ábra. A large-arc-flag paraméter hatása, a szaggatott vonal a másik érték szerinti ív

```
<path d="M 25,50
    A 29.5,50 25 0 1 75,50"
    fill="none" stroke="blue" stroke-width="3">
</path>
```

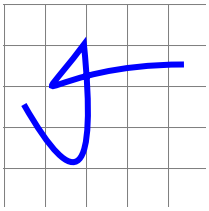


2.12. ábra. A sweep-flag paraméter hatása, a szaggatott vonal a másik érték szerinti ív

Összetett útvonalak

Egy <path> elemen belül az egyenes szakaszok, a Bézier-görbék és az ívek tetszőleges sorrendben követhetik egymást, így szabálytalan alakzatok is leírhatók egyetlen elemmel. A 2.13. ábra egy másodfokú és egy harmadfokú görbét kapcsol össze.

```
<path d="M10,50 Q50,120 40,20 C10,60 20,30 90,30"
    fill="none" stroke="blue" stroke-width="3"/>
```



2.13. ábra. Különböző parancsokból összeállított útvonal kódja és megjelenése

2.1.4. Szöveg és kép

Egy rajz a geometriai alakzatok mellett gyakran tartalmaz szöveget és raszteres képet is.

Szöveg

A szöveget a <text> elem helyezi el. A szöveg karakterekként tárolódik, ezért kijelölhető, kereshető, és nagyításkor is éles marad.

- `x`, `y`: az első karakter alapvonalának kezdőpontja
- `dx`, `dy`: eltolás az előző karakterhez képest
- `rotate`: az egyes karakterek elforgatása
- `fill`, `stroke`: a karakterek kitöltése és körvonala

A betűtípust a `font-family`, a méretet a `font-size` tulajdonság adja meg, a `<tspan>` elem pedig a szöveg egy részének ad eltérő stílust vagy pozíciót. A `font-family` csak a betűtípus nevét rögzíti. Ha a fájlt megnyitó gépen nincs ilyen betűtípus, a böngésző egy másikat választ helyette, és a szöveg szélessége, tördelése és megjelenése megváltozik. Ez kétféleképpen kerülhető el. A betűtípus beágyazható a fájlba, vagy a szöveg útvonallá alakítható. Az utóbbi után a szöveg nem szerkeszthető tovább szövegként, viszont minden környezetben ugyanúgy jelenik meg.

```
<text x="2" y="55" rotate="10" font-size="11"
  fill="gray" stroke="black" stroke-width="0.3">
  Szöveg <tspan fill="blue">kiemelés</tspan>
</text>
```



2.14. ábra. Karakterenként elforgatott szöveg egy eltérő színű `<tspan>` résszel

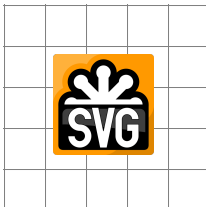
Raszteres kép

Az `<image>` elem raszteres képet, például PNG- vagy JPEG-fájlt helyez el a rajzban.

- `x`, `y`: a kép bal felső sarka
- `width`, `height`: a kép megjelenített mérete
- `href`: a kép címe

A `href` hivatkozhat külső fájlra, de tartalmazhatja magát a képet is base64 kódolású `data:` URL-ként. Az utóbbi esetben az SVG-fájl külső fájlok nélkül is teljes.

```
<image href="svg-logo.png"
    x="25" y="25" width="50" height="50" />
```



2.15. ábra. Külső képfájl elhelyezése

2.1.5. Csoportok, definíciók és színátmenetek

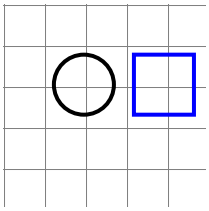
Az eddig bemutatott elemek önállóan is használhatók, egy összetettebb rajzban azonban rendszerint csoportokba szerveződnek, és egyes részeik többször is előfordulnak. Ez az alszakasz a csoportosítást, az újrafelhasználható definíciókat és a definíciókra épülő színátmeneteket mutatja be.

Csoportosítás

A <g> elem [47] összefogja a benne lévő elemeket, így például egy ikon részei egy egységként kezelhetők. A csoport transzformációja minden tagjára érvényes, a megjelenési attribútumait pedig a tagok öröklik, hacsak saját értékkel felül nem írják.

A 2.16. ábrán a csoport fekete körvonalat és eltolást ad meg. A kör örökli a fekete körvonalat, a téglalap saját kék körvonalat kap, az eltolás pedig mindkettőre vonatkozik.

```
<g transform="translate(10,10)"
   stroke="black" stroke-width="2" fill="none">
   <circle cx="30" cy="30" r="15" />
   <rect x="55" y="15" width="30" height="30" stroke="blue" />
</g>
```



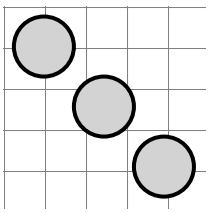
2.16. ábra. Transzformáció és stílusöröklés egy csoportban

Definíciók

A <defs> elem [45] olyan objektumokat tárol, amelyek önmagukban nem jelennek meg, csak hivatkozással használhatók fel. Egy alakzat többszöri megjelenítésére a <use> elem szolgál, a színátmenetekre és a mintákra pedig a `fill` vagy a `stroke` attribútum hivatkozik. A fejezet ábráinak háttérrácsa is egy <defs> elemben definiált minta. A 2.17. ábrán egy egyszer definiált kör három <use> elemmel, három helyen jelenik meg.

```
<defs>
    <circle id="grayCircle" r="15"
    fill="lightgray" stroke="black"
    stroke-width="2"/>
</defs>

<use x="20" y="20" href="#grayCircle" />
<use x="50" y="50" href="#grayCircle" />
<use x="80" y="80" href="#grayCircle" />
```



2.17. ábra. Egy definiált elem többszöri felhasználása

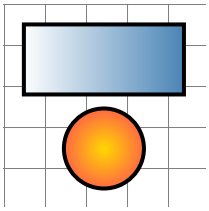
Színátmenetek

A kitöltés és a körvonal egyetlen szín helyett színátmenet is lehet. A színátmenetet egyedi azonosítóval a <defs> elemben kell definiálni, majd a `fill` vagy `stroke` attribútumban `url(#azonosító)` formában lehet rá hivatkozni [75]. Az SVG két színátmenetet ismer. A <linearGradient> lineáris átmenetet ad az `x1`, `y1` és az `x2`, `y2` pont közötti egyenes mentén. A <radialGradient> sugárirányú átmenete a `cx`, `cy` középpontból indul, és az `r` sugárig tart.

Az átmenet színeit <stop> elemek adják meg. Mindegyiknek van egy 0 és 1 közötti `offset` értéke, amely a szín helyét jelöli az átmenet mentén, egy `stop-color` és egy opcionális `stop-opacity` értéke. A koordináták alapértelmezés szerint az

alakzat határoló téglalapjához viszonyítva értendők, ezt a `gradientUnits` attribútum `objectBoundingBox` értéke jelzi. Így ugyanaz az átmenet bármekkora alakzatot ugyanúgy kitölt.

```
<defs>
    <linearGradient id="lin" x1="0" y1="0" x2="1" y2="0">
        <stop offset="0" stop-color="white"/>
        <stop offset="1" stop-color="steelblue"/>
    </linearGradient>
    <radialGradient id="rad" cx="0.5" cy="0.5" r="0.5">
        <stop offset="0" stop-color="gold"/>
        <stop offset="1" stop-color="tomato"/>
    </radialGradient>
</defs>
<rect x="10" y="10" width="80" height="35" fill="url(#lin)"/>
<circle cx="50" cy="72" r="20" fill="url(#rad)"/>
```



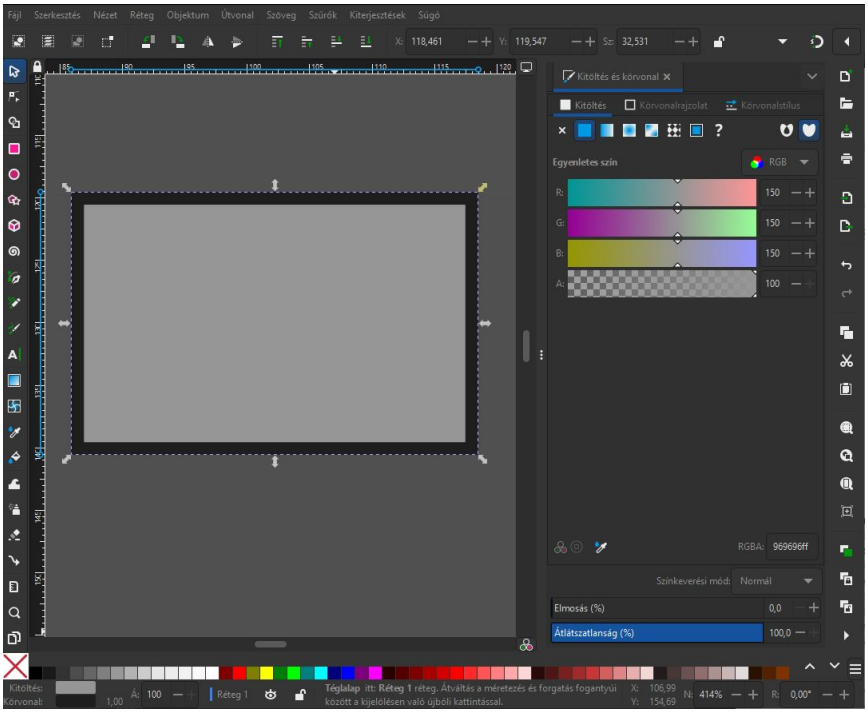
2.18. ábra. Lineáris és sugárirányú színátmenet definiálása és használata

2.2. Szerkesztőeszközök

Az interaktív vektoros rajzolás Ivan Sutherland 1963-as Sketchpad rendszeréig nyúlik vissza, amelyben a rajz elemei már a képernyőn, fényceruzával jelölhetők ki és mozgathatók voltak [63]. A mai SVG-szerkesztők között vannak sok funkciós, de nehezen tanulható programok, és vannak egyszerű, kész elemekből építkező eszközök is. Ebben a szakaszban öt elterjedt asztali és webes alkalmazást mutatok be, majd közös szempontok szerint hasonlítom össze őket. A leírások az alkalmazások dokumentációján, forráskódján és a saját kipróbálásomon alapulnak, és a 2026. novemberi állapotot tükrözik.

A kiválasztott öt programon kívül is léteznek böngészőben futó, nyílt forráskódú eszközök. A Penpot SVG-alapú tervezőeszköz, amely saját szerveren is futtatható, és öt logikai műveletet kínál [60]. A Graphite regisztráció nélkül, helyben fut, és csomópont-alapú, procedurális szerkesztést ad, a dolgozat írásakor azonban még alfa állapotú [25]. Ezeket nem próbáltam ki olyan alaposan, mint a másik ötöt, ezért az összehasonlításban nem szerepelnek.

2.2.1. Inkscape



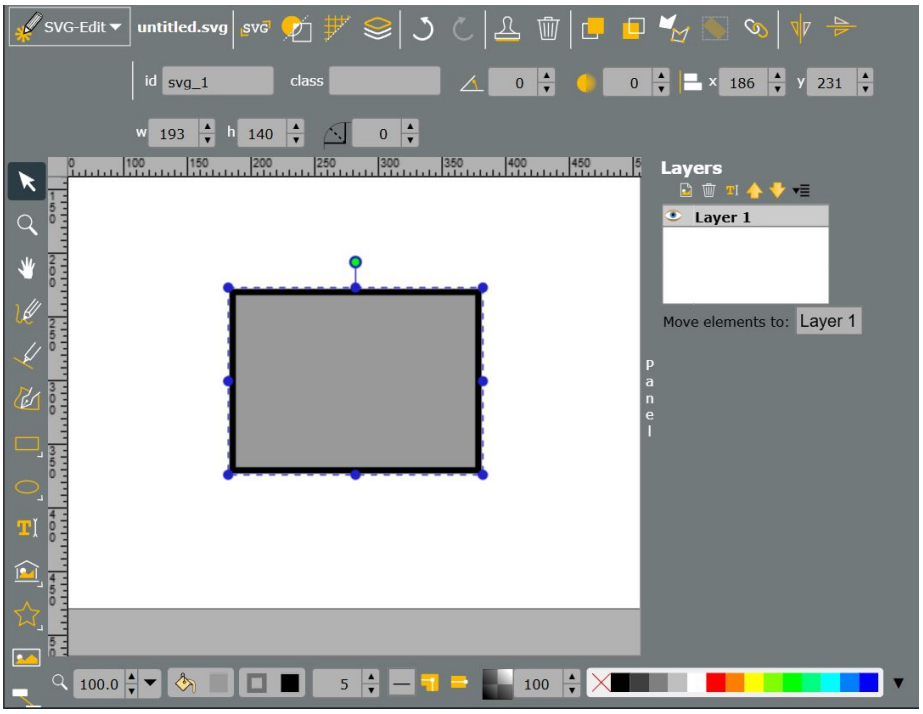
2.19. ábra. Az Inkscape kezelőfelülete [26]

Az Inkscape (2.19. ábra) ingyenes, nyílt forráskódú asztali vektorgrafikus program Windows, macOS és Linux rendszerre [26], alapformátuma az SVG. A felülete sűrű. Bal oldalon az eszköztár, felül a menüsor és az aktuális eszköz beállításait tartalmazó sáv látható, jobb oldalon dokkolható párbeszédablakok, alul a színpaletta és az állapotsor.

Az alapalakzatok mellett csillagot, spirált és szabadkézi vonalakat is rajzol, és minden tulajdonság számszerűen is megadható a párbeszédablakokban. Az XML-szerkesztő fa szerkezetben mutatja a dokumentumot, és bármely attribútum közvetlenül módosítható benne. Az útvonalszerkesztés a program erőssége, a csomóponteszköz minden csomópontot és fogantyút elérhetővé tesz, a csomópontok típusa sarok és sima között váltható. Az útvonalakon unió, különbség, metszet, kizárás és felosztás végezhető, emellett útvonal elvágására és összevonására is van parancs [27]. Az Alakzatépítő eszköz egymást metsző alakzatokból kattintásokkal állít össze új alakzatot [28].

A szöveg a vásznon és egy külön szövegablakban is szerkeszthető, útvonalra illeszthető, a betű- és sorköz és az alávágás is állítható. A színek RGB, HSL, HSV, CMYK és OKHSL színmódban adhatók meg [28]. A kitöltés lehet lineáris, sugárirányú vagy hálós színátmenet és minta is. A rétegek és objektumok egy fában jelennek meg, ahol zárolhatók és elrejthetők, az illesztés pedig a rácshoz, a segédvonalakhoz és a más objektumok jellemző pontjaihoz egyaránt beállítható. Az előzmények külön panelen láthatók, az automatikus mentés pedig a beállításokban szabályozható.

2.2.2. SVG-Edit



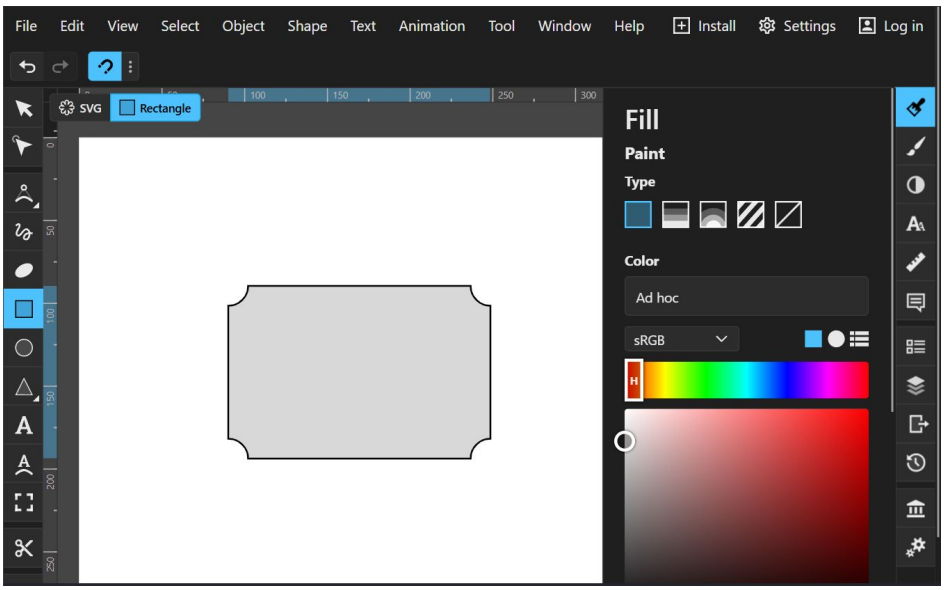
2.20. ábra. Az SVG-Edit kezelőfelülete [67]

Az SVG-Edit (2.20. ábra) nyílt forráskódú, MIT licencű webes szerkesztő, amely regisztráció nélkül, közvetlenül a böngészőben fut [67, 66]. A felület tetején a kijelölt elem beállításai, bal oldalon az eszköztár, alul a kitöltés és a körvonal vezérlői, jobb oldalon a rétegpanel található.

Az alapalakzatok mellett egy beépített alakzatkönyvtárat is kínál. A forráskód-szerkesztő ablakban a teljes SVG-kód módosítható, a változtatások a módosítások alkalmazása után jelennek meg a vásznon. Az útvonalak csomópontjai mozgathatók, törölhetők és klónozhatók, a szakaszok típusa egyenes és görbe között váltható, a két kontrollpont pedig összekapcsolható. Logikai műveleteket nem kínál [66].

A szöveg a vásznon szerkeszthető, a betűstílus, a díszítővonalak, a betű- és szóköz, valamint az igazítás állítható. A beviteli mező csak egysoros szöveget fogad, és nyolc betűcsalád közül lehet választani [66]. A színválasztó HEX, RGB és HSV értékeket fogad átlátszósággal együtt, a kitöltés egyszínű, lineáris vagy sugárirányú színátmenet lehet. Az illesztés a rácshoz kapcsolható be, az igazító parancsok több elemre is működnek. A visszavonás és a megismétlés elérhető, egy bővítmény pedig a lap bezárásakor a böngésző helyi tárolójába menti a rajzot, ha a felhasználó ezt engedélyezi.

2.2.3. Boxy SVG



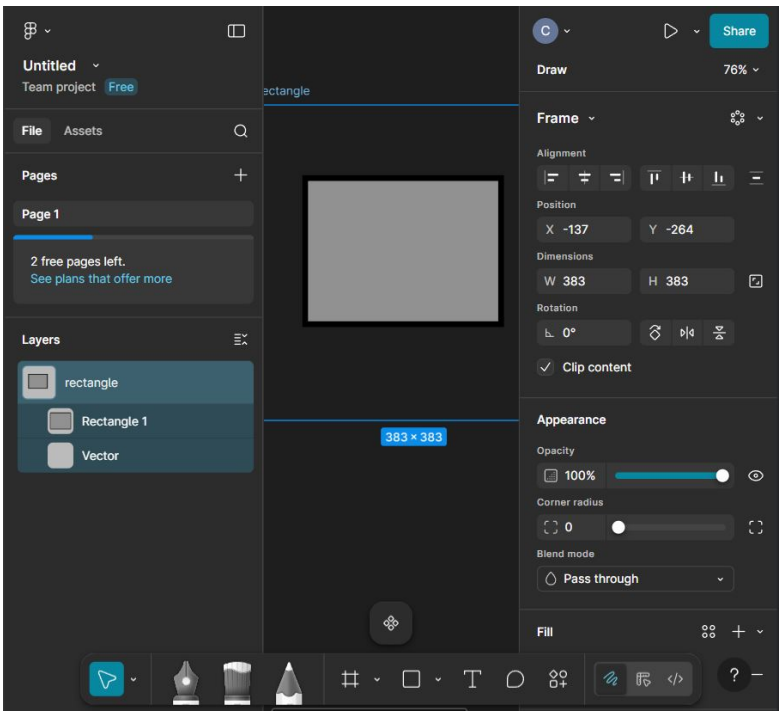
2.21. ábra. A Boxy SVG kezelőfelülete [2]

A Boxy SVG (2.21. ábra) zárt forráskódú, freemium [3] vektorgrafikus szerkesztő, vagyis az alapváltozata ingyenes, a további szolgáltatásai pedig fizetősek. Webalkalmazásként és asztali alkalmazásként is elérhető. A webalkalmazás alapvető szerkesztő- és exportálási funkciói regisztráció és fizetés nélkül is használhatók, az asztali alkalmazások egy része és a haladó funkciók, például az automatikus felhőszinkronizálás és a teljes verziótörténet, viszont előfizetéshez kötöttek [2]. A felület tetején menüsor van, alatta egy sáv a csoportosításhoz, igazításhoz, forgatáshoz és a logikai műveletekhez. A vásznat vonalzók veszik körül, a tulajdonságok a jobb oldali sávból nyíló panelekben állíthatók.

Az alapelemek mellett gyakori alakzatok, például háromszög és csillag is rajzolhatók. Az útvonalakhoz szabadkézi, másod- és harmadfokú görberajzoló eszköz tartozik, a csomópontok fogantyúkkal szerkeszthetők. Az alakzatokon unió, különbség, metszet, kizárás és felosztás végezhető. Az Elemek panel a dokumentum szerkezetét és SVG-kódját mutatja, a kód közvetlenül szerkeszthető, és az animációk kulcskockái is szerkeszthetők [2].

A szöveg a vásznon szerkeszthető és útvonalra illeszthető, a betűtípusok a Google Fonts gyűjteményéből, a gépre telepített vagy fájlból betöltött betűtípusok közül választhatók. A színek tizenkét színtérben adhatók meg, a szabványos és a széles színtartományú RGB-terektől az emberi színérzékelést követő CIE- és OK-modellekig, egy szöveges mezőben pedig HEX kóddal is. A kitöltés lineáris és sugárirányú színátmenet vagy minta is lehet, a gyakran használt színek, színátmenetek és minták a `<defs>` elemben tárolhatók. Az elrendezést segédvonalak, intelligens segédvonalak és rács segítik, a visszavonható lépések pedig előzménypanelen láthatók.

2.2.4. Figma



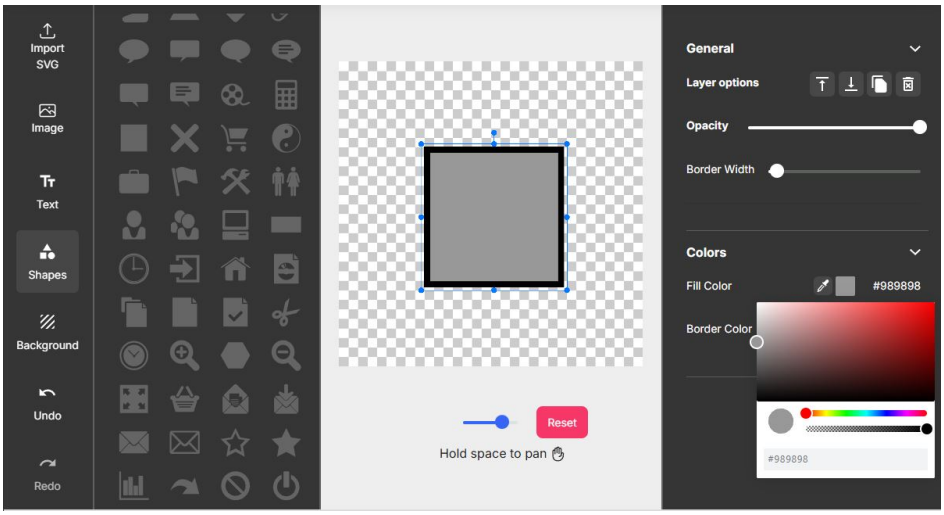
2.22. ábra. A Figma kezelőfelülete [15]

A Figma (2.22. ábra) böngészőben és asztali alkalmazásként is futó felülettervező eszköz, amely szintén freemium konstrukcióban, ingyenes alapcsomaggal és előfizetéses csomagokkal érhető el [15]. A rajzokat saját formátumban tárolja, az SVG számára csak importálási és exportálási formátum. A felület közepén a vászon van, bal oldalon a rétegek, jobb oldalon a tulajdonságok, az eszköztár pedig a vászon alján található [16].

Egy menüből téglalap, vonal, nyíl, ellipszis, sokszög, csillag és kép helyezhető el [11]. Az útvonalak helyett a Figma vektorhálózatokkal dolgozik, amelyekben egy csomópontból kettőnél több szakasz is indulhat [17]. Az unió, a különbség, a metszet és a kizárás roncsolásmentes, az eredmény egy logikai csoport, amelynek tagjai később is módosíthatók [13]. SVG-kódot a szerkesztő nem mutat, és az közvetlenül nem is módosítható.

A Google Fonts betűtípusai alapértelmezés szerint elérhetők, a gépre telepített betűtípusokhoz egy segédprogramra van szükség [12]. A színválasztó HEX, HSB, HSL, CSS és RGB formátumot ismer [14], a kitöltés lehet egyszínű, színátmenet, minta, kép vagy videó. A rétegek fa szerkezetben jelennek meg. A munka folyamatosan a felhőbe mentődik, a verziótörténet az ingyenes csomagban 30 napra visszamenőleg érhető el [18].

2.2.5. Mediamodifier



2.23. ábra. A Mediamodifier kezelőfelülete [54]

A Mediamodifier (2.23. ábra) online grafikai és termékbemutató-készítő szolgáltatás, amely egy SVG-szerkesztőt is kínál [54]. A szerkesztés ingyenes, a kész munka letöltéséhez azonban előfizetés szükséges. Egyszerű felületén bal oldalon az eszközsáv, jobb oldalon a tulajdonságok, középen a vászon található.

A szerkesztő kész elemek összeállítására épül. SVG-fájlok húzással helyezhetők a vászonra és kombinálhatók, az elemek csoportosíthatók, csoportbontással egyenként módosíthatók, és előrébb vagy hátrébb hozhatók [54]. A kitöltés és a körvonal színe, valamint a körvonal vastagsága állítható. A kipróbálás során az alapalakzatok kész matricaként viselkedtek, a görbék rajzolására és az útvonalak szerkesztésére nem találtam lehetőséget, és a beimportált fájlok közül csak az egyszerű alakzatokból állók jelentek meg hibátlanul. A szöveg a vásznon szerkeszthető, de csak a betűtípus és a szín választható meg. Az SVG-kód nem érhető el.

2.2.6. Összehasonlítás

Az összehasonlítás szempontjait úgy választottam meg, hogy egy SVG-szerkesztő mindennapi használatát fedjék le. A felület elrendezésénél azt vizsgáltam, hogy az eszközök a megszokott helyen, logikusan csoportosítva érhetők-e el, és hogy a tulajdonságok számszerűen is megadhatók-e. Fejlesztői szempontból fontos, hogy a szerkesztő megmutatja-e az előállított SVG-kódot, és módosítható-e közvetlenül.

A rajzolás és szerkesztés szempontjai az útvonalakra összpontosítanak: hozzáadhatók és törölhetők-e a csomópontok, kényelmesen mozgathatók-e a fogantyúk, és elérhetők-e a logikai műveletek, vagyis az unió, a különbség, a metszet, a kizárás és a felosztás. Ide tartozik az illesztés a rácshoz és más objektumokhoz is. A szövegnél a többsoros bevitel, a vásznon történő szerkesztés és a betűtípusok forrása számít, a színeknél a színmegadás módjai, az átlátszóság, a színátmenetek és a minták.

A dokumentum kezelésénél a rétegek és csoportok megjelenítését, az előzményeket és a mentést vizsgáltam. A kijelölés, a nagyítás és a vágólap alapműveletei minden vizsgált szerkesztőben működtek, ezért ezeket a táblázat külön nem tünteti fel. A 2.2. táblázat az eredményt foglalja össze.

2.2. táblázat. A vizsgált SVG-szerkesztők összehasonlítása

	Inkscape	**SVG-Edit**	**Boxy SVG**	**Figma**	**Mediamodifier**
Platform, licenc	asztali, nyílt forráskódú	webes, nyílt forráskódú	webes és asztali, freemium	webes és asztali, freemium	webes, letöltés előfizetéssel
Elrendezés	klasszikus, dokkolható panelekkel	klasszikus, alsó stílussávval	klasszikus, beúszó panelekkel	vászon középen, eszköztár alul	egyszerű, két oldalsávval
SVG-kód	XML-szerkesztő	forrásablak, alkalmazás után frissül	Elemek panel, szerkeszthető	nincs	nincs
Szöveg	többsoros, útvonalra illeszthető	egysoros, 8 betűcsalád	útvonalra illeszthető, Google Fonts	Google Fonts, helyi betűtípusok	betűtípus és szín
Színek	RGB, HSL, HSV, CMYK, OKHSL	HEX, RGB, HSV, átlátszóság	12 színtér (RGB, CIE, OK), HEX	HEX, HSB, HSL, CSS, RGB	színválasztó
Színátmenet, minta	lineáris, sugárirányú, hálós; minta	lineáris, sugárirányú	lineáris, sugárirányú; minta	színátmenetek, minta, kép	egyszínű
Útvonalak	teljes csomópont-szerkesztés	csomópontok, szakasztípus	csomópontok, fogantyúk	vektorhálózatok	nincs
Logikai műveletek	unió, különbség, metszet, kizárás, felosztás	nincs	unió, különbség, metszet, kizárás, felosztás	unió, különbség, metszet, kizárás	nincs
Illesztés	rács, segédvonalak, objektumok	rács	rács, segédvonalak	igazítási segédvonalak	alapszintű
Rétegek	objektumfa, zárolás	rétegpanel	elemfa	rétegfa	előre-hátra mozgatás
Előzmények, mentés	előzménypanel, automatikus mentés	visszavonás, mentés a lap bezárásakor	előzménypanel, felhőtárhely	verziótörténet, folyamatos felhőmentés	letöltés előfizetéssel

2.2.7. Következtetések és célkitűzések

A vizsgált szerkesztők közül az Inkscape kínálja a legteljesebb SVG-szerkesztést, de asztali program, amelyet telepíteni kell. A Boxy SVG hasonlóan sokoldalú, böngészőben is fut, haladó funkciói azonban előfizetéshez kötöttek, és a forráskódja zárt. A Figma kiváló vektoros eszközei saját adatmodellre épülnek, az SVG csak be- és kimeneti formátum. A Mediamodifier kész elemek összeállítására készült, görbéket nem lehet vele rajzolni. Az SVG-Edit ingyenes és nyílt forráskódú, kódnézetet is kínál, de csak egysoros szöveget kezel, és logikai műveletek nincsenek benne.

Ezek alapján a saját szerkesztővel szemben a következő célokat tűztem ki:

1. A szerkesztő szerver, telepítés és regisztráció nélkül, ingyenesen fusson a böngészőben, klasszikus elrendezéssel.
2. Kínáljon a vizsgált szerkesztőkhöz mérhető rajzoló és szerkesztő eszközöket, köztük teljes értékű útvonalszerkesztést és logikai műveleteket, amelyek eredménye görbékből álló, tovább szerkeszthető alakzat marad.
3. Kezeljen többsoros szöveget, és alakítsa útvonallá úgy, hogy az eredmény megegyezzen a képernyőn látottal.
4. Importáljon SVG-fájlokat, és jelezze, milyen szerkezeteket nem tud kezelni.
5. Exportáljon szabványos SVG-fájlt és raszteres képet.
6. Legyen telefonon és táblagépen is kezelhető.

A 3. fejezet ezeket a célokat bontja részletes követelményekre, a 6. fejezet pedig azt értékeli, mennyiben teljesültek.

3. fejezet

Tervezés

A fejezet a saját szerkesztővel szemben támasztott követelményekkel kezdődik, amelyek a 2.2.6. alszakasz szempontjait követik. Ezután a felület terve, a felhasznált technológiák, az alkalmazás rétegei, az adatmodell, az előzmények kezelése, az eszközök közös felülete és a tárolás következik.

3.1. Követelmények

A 2.2.7. alszakasz célkitűzései még csak az irányt jelölik ki. Ez a szakasz ezeket ellenőrizhető funkcionális és nem funkcionális követelményekre bontja.

3.1.1. Funkcionális követelmények

A 3.1. táblázat a funkcionális követelményeket sorolja fel. A területek nagyrészt megegyeznek a 2.2. táblázat soraival, így szempontonként összevethető, melyik megoldást követi a saját szerkesztő. Az utolsó oszlop azt mutatja, melyik célkitűzéshez tartozik az adott követelmény. Az azonosítókra a tesztelés és az értékelés is hivatkozik.

Az F4 követelmény külön indoklást kíván, mert a szerkesztő nem tartalmaz kódszerkesztőt, holott a 2.2. szakaszban vizsgált programok közül három is kínál ilyet. A szerkesztő saját adatmodellel dolgozik, amelyet a 3.5. szakasz ír le, egy élő kódszerkesztőnek tehát minden begépelt változtatást ellenőriznie és erre a modellre kellene alakítania. Ez lényegében az importáló folyamatos futtatását jelentené, ezért a kódnézetet a továbbfejlesztési lehetőségek közé soroltam.

3.1.2. Nem funkcionális követelmények

Az első célkitűzésből következik, hogy az alkalmazás kizárólag a böngészőben futhat. A fájlok a felhasználó gépén maradnak, hálózatra csak a Google Fonts betűtípusainak letöltéséhez van szükség. A felületnek asztali gépen, táblagépen és telefonon is használhatónak kell lennie. Keskeny kijelzőn a tulajdonságpanel nem takarhatja el a rajzot, és az érintéssel használt fogantyúknak elég nagynak kell lenniük.

A felület tervezésekor Nielsen tíz használhatósági heurisztikája közül négyre figyeltem különösen [29]. A rendszer állapotának láthatóságát az elnevezett előzménylépések szolgálják, a felhasználó szabadságát pedig az, hogy bármely korábbi állapotba visszaléphet. A megszokott konvenciókhoz igazodnak a más grafikus programokéval egyező billentyűkombinációk. A felismerés előnyben részesítése az emlékezéssel szemben a

3.1. táblázat. Funkcionális követelmények

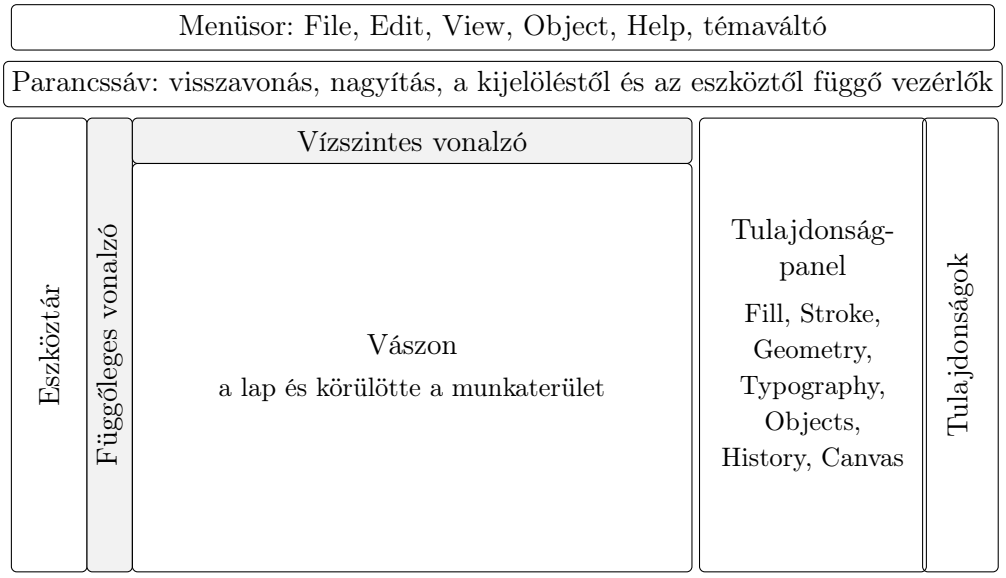
Az.	Terület	Követelmény	Cél
F1	Elrendezés	Felül menüsor és a helyzethez igazodó parancssáv, bal oldalon eszköztár, középen vonalzós vászon, jobb oldalon tulajdonságpanel. Keskeny kijelzőn a panel alulról nyílik.	1, 6
F2	Rajzolás	Téglalap, ellipszis, vonal, sokszög és töröttvonal, Bézier-görbéket rajzoló toll, körív, szabadkézi rajz és szöveg, csoportosítva az eszköztáron.	2
F3	Tulajdonságok	Minden alakzat módosítható a vásznon fogantyúkkal és a panelen számszerűen, elforgatva is. Több kijelölt alakzat együtt mozgatható és méretezhető.	2
F4	SVG-kód	Kódszerkesztő nincs, a kimenet szabványos, olvasható SVG.	5
F5	Szöveg	Többsoros szöveg a vásznon szerkesztve. Betűtípus, méret, vastagság, dőlt stílus, díszítővonalak, betű- és szóköz, sorköz és igazítás. Útvonallá alakítás a képernyőn látottal egyező eredménnyel.	3
F6	Színek	HEX, RGB és HSL színmegadás átlátszósággal, lineáris és sugárirányú színátmenet, szaggatás, vonalvég, sarok és kitöltési szabály.	2
F7	Útvonalak	Csomópontok hozzáadása, törlése, mozgatása és többes kijelölése, váltás sarok és sima csomópont között, a fogantyúk mozgatása. Unió, különbség, metszet, kizárás és felosztás, alakzatok vágása, körvonal útvonallá alakítása.	2
F8	Illesztés, igazítás	Illesztés rácshoz, más alakzatok széleihez és középvonalaihoz, a vonalzókból húzott segédvonalakhoz. Igazító parancsok több alakzatra.	2
F9	Objektumok	Csoportosítás és belépés a csoportba, az alakzatok fája, a sorrend módosítása húzással, elrejtés és zárolás.	2
F10	Kijelölés	Kattintás, bővítés Shift billentyűvel, területkijelölés. Érintőképernyőn a Shift helyett bekapcsolható gomb.	2, 6
F11	Vágólap	A szokásos billentyűkombinációk, másolás más programokba és más programokból, valamint a szerkesztő böngészőlapjai között.	2
F12	Nagyítás	Görgővel a kurzor körül, gombokkal, a lapra illesztve, érintőképernyőn két ujjal.	2, 6
F13	Visszavonás	Minden módosítás visszavonható és megismételhető, a lépések névvel jelennek meg. Egy folyamatos állítás egyetlen lépés.	2
F14	Mentés	Folyamatos automatikus mentés a böngészőbe és visszatöltés újranyitáskor, mentés projektfájlba.	1
F15	Import, export	SVG-fájlok és raszteres képek importálása, a nem támogatott szerkezetekről figyelmeztetés. Exportálás SVG, PNG, JPG és WebP formátumba.	4, 5

parancssávban valósul meg, amely mindig az adott helyzetben elérhető műveleteket mutatja.

Egy sérült vagy szándékosan rosszul összeállított fájl nem teheti használhatatlanná a szerkesztőt, ezért a betöltött projektfájlokat mezőnként ellenőrizni kell, és a hibás részeket el kell dobni. Végül a geometriai számításokat, az előzményeket és a fájlkezelést automatikus teszteknek kell ellenőrizniük.

3.2. A felhasználói felület terve

A felület tervezésekor az elrendezésből indultam ki, amelyet a 3.1. ábra mutat. A szerkesztő angol nyelvű, ezért az ábrán és a későbbi képernyőképeken angol feliratok láthatók, a szöveg pedig ezek magyar megfelelőit használja.



3.1. ábra. A felület elrendezése asztali kijelzőn

A menüsor a ritkábban használt, de mindig elérhető parancsokat gyűjti össze: a fájlkezelést, a szerkesztési műveleteket, a nézet beállításait, az objektumokra vonatkozó műveleteket és a súgót. A parancssáv bal szélén mindig ott van a visszavonás, a megismétlés és a nagyítás, a többi vezérlője az aktuális eszköztől és a kijelöléstől függ. Egy kijelölt alakzatnál a forgatás, az átlátszóság és a tükrözés jelenik meg, több alakzatnál az igazítás és a logikai műveletek, a tollnál pedig egy rövid útmutató a használatához.

A bal oldali eszköztár tizenhárom eszköze hat csoportot alkot. Az első a kijelölés és a csomópontok szerkesztése, a második a vonalak és görbék rajzolása, a harmadik a zárt alakzatoké. Ezt követi a szöveg, majd a meglévő alakzatokon dolgozó foltecset és olló, végül a nézet mozgatása. Minden eszköznek saját billentyűje van, ezeket a súgó is felsorolja.

A jobb oldali tulajdonságsáv hét panelt nyit meg. A kitöltés és a körvonal panel a színeket és a vonal stílusát állítja. A geometria panelen a pozíció, a méret, a forgatás és a forgatási középpont adható meg, a tipográfia panelen a szöveg beállításai. Az objektumok panel a dokumentum fáját mutatja, az előzmények panel a visszavonható lépéseket, a vászon panel a lap méretét, hátterét, a rácsot és az illesztést. A panel szélessége húzással állítható, a nyitott panel fülére újra kattintva pedig a panel bezárul, és a vászon a teljes szélességet megkapja.

Keskeny kijelzőn a panel már nem férne el a rajz mellett. Ilyenkor a vászon alatt, alulról nyíló lapként jelenik meg, amelynek magassága a felső szélénél húzva állítható. Az eszköztár és a tulajdonságsáv a helyén marad, így nyitott panel mellett is elérhetők.

A műveletek többféle úton érhetők el, de egyik sem csak rejtett úton. A gyakoriak a parancssávban vannak, a helyzethez kötöttek a jobb gombbal vagy hosszú érintéssel nyíló helyi menüben, és a menüsor mindegyiket tartalmazza. A helyi menü csak az adott helyzetben végrehajtható műveleteket sorolja fel, mert a letiltott menüpontok nehezebben áttekinthetővé tennék.

3.3. Technológiai választások

Az alkalmazás teljes egészében kliensoldali webalkalmazás, a technológiák kiválasztásakor ezért a böngészőben futó, szerver nélkül is használható megoldásokat kerestem. Ez a szakasz a keretrendszert, a fejlesztői eszközöket és a külső könyvtárakat mutatja be, a 3.2. táblázat pedig a pontos verziókat is rögzíti.

3.3.1. Svelte és TypeScript

A Svelte komponensei egyetlen fájlban írják le a szerkezetet, a stílust és a viselkedést, a fordító pedig ezekből a DOM-ot közvetlenül módosító JavaScript kódot állít elő [64]. Az alkalmazás állapotát Svelte *store*-ok tárolják. A store olyan objektum, amelyre a komponensek feliratkozhatnak, és amely minden változásról értesíti őket [65]. A származtatott store más store-ok értékéből számol, és csak akkor frissül, ha a forrása megváltozott. Egy szerkesztőben ugyanazt a dokumentumot a vászon, a panelek és az objektumlista egyszerre jeleníti meg, és ez a modell mindhármat automatikusan naprakészen tartja.

Általános komponenskönyvtárat, például a Bootstrapet, nem használtam. A szerkesztő vezérlői, a színválasztó, a csúszkák, a fogantyúk és az objektumlista, egyedi elemek, egy komponenskönyvtár pedig saját megjelenést és elrendezési szabályokat hozott volna, amelyek a sűrű szerkesztőfelülethez kevéssé illeszkednek.

A TypeScript használatát az adatmodell indokolja. Az alakzatok típusa egy közös típusmező szerint elágazó unió, angol nevén *discriminated union*, és a típusellenőrzés segít észrevenni, ha egy függvény valamelyik alakzattípus kezelését kihagyja.

3.3.2. Fejlesztői eszközök

A projekt a Vite fejlesztői környezetet használja. Fejlesztés közben a Vite a forrásfájlokat natív ES modulokként szolgálja ki, ezért egy módosítás után elég az érintett modult újratölteni [69]. Az éles változatot is a Vite csomagolja statikus fájlokká, amelyeket bármely webszerver kiszolgálhat.

A tesztek a Vitest keretrendszerrel futnak, amely a Vite beállításait használja, a komponensek teszteléséhez pedig a jsdom könyvtár ad böngészőszerű környezetet Node.js alatt [70, 31]. A böngészőben végzett ellenőrzések egy részét a Playwright eszközzel automatizáltam, amely programból vezérel egy Chromium böngészőt [55].

3.3.3. Geometriai és betűtípus-könyvtárak

A geometriai algoritmusok többsége saját kód, három területen azonban külső könyvtárra támaszkodik a szerkesztő: a logikai műveleteknél és a vágásnál a paper.js, a körvonal útvonallá alakításánál és a foltecsetnél a Clipper, a szöveg útvonallá alakításánál pedig a fontkit könyvtárra. A 3.2. táblázat a fejlesztői eszközökkel együtt sorolja fel őket.

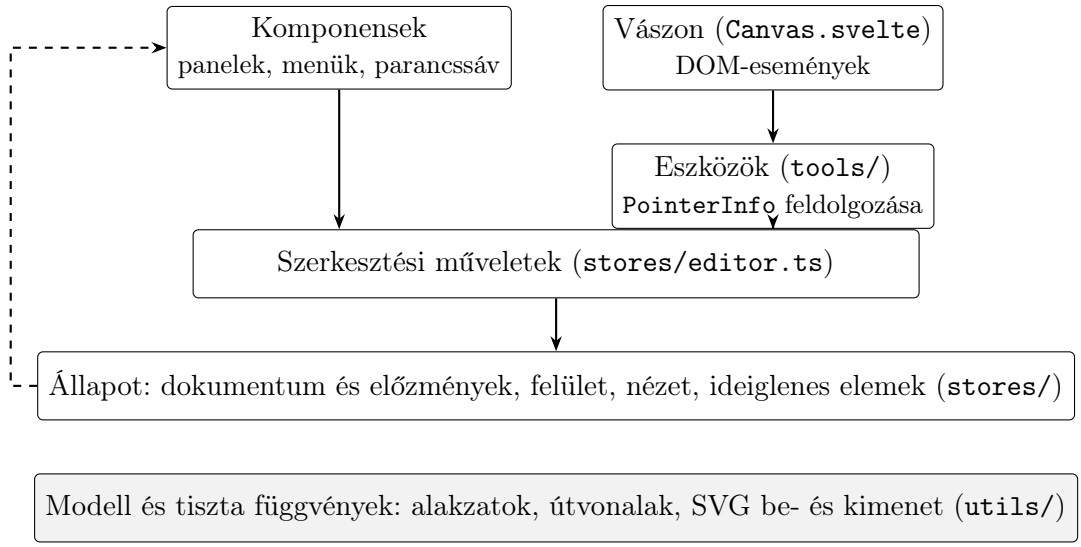
3.2. táblázat. A felhasznált eszközök és könyvtárak

Név	**Verzió**	**Szerep**
Svelte	5.57.2	felhasználói felület, állapotkezelés
TypeScript	5.9.3	programozási nyelv
Vite	7.3.7	fejlesztői szerver, éles változat csomagolása
Vitest	3.2.7	automatikus tesztek
jsdom	26.1.0	böngészőszerű környezet a tesztekhez
paper.js	0.17.0	logikai műveletek, alakzatok vágása
clipper-lib	6.4.2	körvonal útvonallá alakítása, foltecset
fontkit	2.0.4	szöveg útvonallá alakítása, OpenType-szövegszedés
woff2-encoder	2.0.0	WOFF2 betűtípusfájlok kibontása a fontkit számára

A sokszögek logikai műveleteire több klasszikus algoritmus ismert. Vatti söprővonalas algoritmusa tetszőleges, akár önmagukat metsző sokszögeket is kezel [68], ennek továbbfejlesztése a Clipper könyvtár [30]. A Martínez-Rueda-Feito algoritmus $O((n + k) \log n)$ időben dolgozik, ahol n a csúcsok, k a metszéspontok száma [33], ezt valósítja meg a polygon-clipping könyvtár [19]. A logikai műveletekhez a polygon-clipping és a clipper2-ts könyvtárat is megvizsgáltam. Mindkettő sokszögekkel dolgozik, a görbéket előbb rövid egyenes szakaszokra bontják. Két kör egyesítéséből így több száz sarokpontú alakzat lenne, amely kézzel már alig szerkeszthető. A paper.js a görbék metszéspontjait számolja ki, és az eredményt is görbékből építi fel [32]. A körvonal útvonallá alakítása és a foltecset eredménye eleve sokszög, ezekhez a Clipper eltolási és egyesítési műveleteit használtam [30].

3.4. Az alkalmazás rétegei

Az alkalmazás négy rétegből áll, ezeket és a köztük haladó adatfolyamot a 3.2. ábra mutatja.



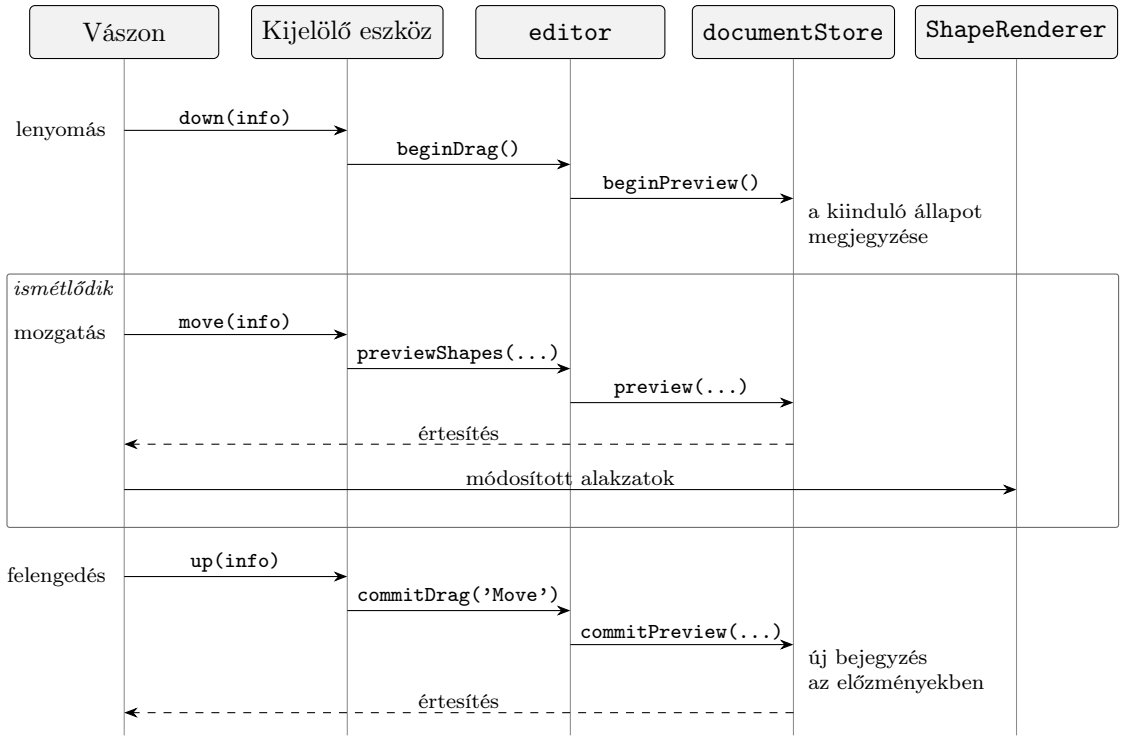
3.2. ábra. Az alkalmazás rétegei és az adatfolyam iránya; a szaggatott nyíl a store-ok értesítéseit jelöli

A legalsó réteg az `utils` mappa. Itt van a dokumentum típusainak leírása és a mellékhatás nélküli, úgynevezett tiszta függvények: a határoló téglalapok számítása, az alakzatok mozgatása és méretezése, az útvonalak feldolgozása, az SVG importálása és előállítása. Ezek a függvények sosem módosítják a kapott alakzatot, hanem új alakzatot adnak vissza. Fölöttük a `stores` mappa store-jai tárolják a dokumentumot és az előzményeit, a felület állapotát a kijelöléssel és az aktuális eszközzel, a nézet nagyítását és eltolását, valamint az ideiglenes rajzelemeket, például a rajzolás közbeni előnézetet.

A dokumentumot csak az `editor` objektum műveletei módosítják. Ilyen az alakzat hozzáadása, a stílus módosítása, a csoportosítás, az igazítás, a logikai művelet és a beillesztés, és mindegyik egy elnevezett, visszavonható lépést hoz létre. A legfelső rétegben a rajzoló eszközök a vászontól kapják a mutató eseményeit, a komponensek pedig a felhasználó kattintásait. Mindkettő csak az `editor` műveleteit hívja, az eredményt pedig a store-okból olvassa vissza.

Ennek az egyirányú adatfolyamnak két haszna van. Egy művelet ugyanúgy viselkedik, akár a panelről, akár a menüből, akár a helyi menüből indítják. Az eszközök és a műveletek emellett böngésző nélkül is tesztelhetők, mert egy eszköz kipróbálásához elég mesterséges mutatóeseményeket átadni neki, majd megvizsgálni a dokumentumot.

A 3.3. ábra ezt az adatfolyamot egy alakzat húzással történő mozgatásán mutatja be. A kijelölő eszköz a lenyomáskor előnézeti módba kapcsolja a dokumentumot, mozgatás közben csak az előnézetet frissíti, a felengedéskor pedig egyetlen elnevezett lépésként rögzíti a változást. A vászon a store-ra feliratkozva értesül minden módosításról, és ennek hatására rajzolja újra az alakzatokat.



3.3. ábra. Egy alakzat mozgatásának lépései a rétegeken át; a szaggatott nyilak a store értesítéseit jelölik

3.5. Adatmodell

A dokumentum JSON formában tárolható adatszerkezet. Az alakzatok fáján kívül a lap méretét, a hátteret, egy opcionális háttérképet és a vonalzókból húzott segédvonalakat tartalmazza. Az alakzatok típusát a `type` mező adja meg, és ez dönti el, milyen geometriai adatok tartoznak hozzájuk. A 3.3. táblázat az alakzattípusokat és a kimenetben nekik megfelelő SVG-elemeket mutatja.

3.3. táblázat. Az alakzattípusok, geometriai adataik és SVG megfelelőjük

Típus	**Geometria**	**SVG-elem**
`rectangle`	bal felső sarok, szélesség, magasság, saroksugarak	`<rect>`
`ellipse`	középpont, két sugár	`<ellipse>`
`line`	két végpont	`<line>`
`path`	csomópontok be- és kimenő fogantyúval, zártság, kitöltési szabály	`<path>`
`text`	kezdőpont, sorok, betűtípus, díszítővonalak, térközök	`<text>`
`image`	bal felső sarok, méret, a kép `data:` URL-je	`<image>`
`group`	a csoport tagjai	`<g>`

Az alakzatok közös adatai az egyedi azonosító, név, láthatóság, zárolás, átlátszóság, kitöltés, körvonal és vonalvastagság, vonalvég, sarok, szaggatási minta és forgatás.

Útvonalak. A szerkesztő az útvonalat csomópontok listájaként tárolja, az SVG `d` attribútumának parancsait csak importáláskor és exportáláskor használja. Egy csomópontnak van helye, opcionális bejövő és kimenő fogantyúja, és egy jelzője arról, hogy sima-e. Két csomópont között egyenes szakasz van, ha a köztük lévő oldalon egyiknek sincs fogantyúja, különben harmadfokú Bézier-görbe. Ebben a formában minden csomópont és fogantyú közvetlenül megfogható. Az SVG másodfokú görbéit és íveit az importáló harmadfokú görbékké alakítja.

Színek és színátmenetek. A kitöltés és a körvonal értéke egy karakterlánc. Lehet `none`, egy `#rrggbb` színkód vagy átlátszósággal együtt `#rrggbbaa`, illetve egy színátmenet leírása. Egy lineáris átmenet például `lg:90;#1e3c72@0,#f7b267@100` alakú. Az első szám a szög fokban, utána a színek következnek a helyükkel együtt, százalékban. A színek és a színátmenetek így ugyanazon a kódon haladnak át stílus másolásakor, összehasonlításkor vagy visszavonáskor. Ha a színátmenet külön objektum lenne, ezeket a műveleteket két változatban kellene megírni. Exportáláskor minden színátmenet egy `<defs>` elembe kerül, és az alakzat `url()` hivatkozással használja.

Csoportok. A csoport forgatása és átlátszósága a csoport egészére vonatkozik, a tagok koordinátái pedig úgy értendők, mintha a csoport nem lenne elforgatva. A felhasználó beléphet egy csoportba, ekkor a kijelölés és a szerkesztés a csoport tagjaira vonatkozik.

Forgatás és forgatási középpont. Az alakzat forgatása fokban, a saját, el nem forgatott határoló téglalapjának közepe körül értendő, az exportált fájlban ebből lesz a `rotate(szög cx cy)` transzformáció. Ehhez járul egy opcionális forgatási középpont, amely a határoló téglalaphoz viszonyított arányként adja meg, mely pont körül forog az alakzat a szerkesztőben. A forgatási középpont csak szerkesztési segédlet. Forgatáskor a szerkesztő a szöget és a pozíciót együtt módosítja úgy, hogy a kijelölt pont a helyén maradjon, a tárolt forgatás pedig mindig a téglalap közepe körüli marad. Erre a feltevésre épül a határoló téglalap, a méretezés, a tükrözés és az export minden számítása, egy valódi, tetszőleges forgatási középpont mindegyiket bonyolultabbá tenné.

3.6. Az előzmények kezelése

A visszavonásra két elterjedt tervezési minta létezik [22]. A *Command* minta minden műveletet olyan objektumként tárol, amely a végrehajtást és a visszacsinálást is ismeri. Berlage ezt a megoldást grafikus felületekre a tetszőleges korábbi lépés visszavonásáig, a szelektív visszavonásig fejlesztette tovább [1]. A *Memento* minta ehelyett az állapotról készít pillanatképet. A szerkesztő a második megoldást követi. A dokumentum adatai sosem módosulnak helyben, minden művelet új dokumentumot készít, amely a változatlan alakzatokat megosztja az előzővel. Egy pillanatkép ezért csak a megváltozott alakzatok helyét foglalja, a visszavonás pedig egyetlen hivatkozás cseréje. Ez a perzisztens adatszerkezetek elve, amelyekben a módosítás új változatot hoz létre, a régi változatok elérhetők maradnak, és a változatok a közös részeiket megosztják [8, 58]. A legfontosabb előnye, hogy egy új művelethez nem kell külön visszacsináló kódot írni, és így ilyen kód hibájából sem állhat vissza rossz állapot.

Az előzmények a korábbi állapotokból, a jelenlegi állapotból és a visszavont, de megismételhető állapotokból álló idővonalat alkotnak. Minden bejegyzés a nevében szereplő művelet utáni állapotot tárolja. Az előzmények panelen egy bejegyzésre kattintva tehát pontosan az a rajz jelenik meg, amelyet a bejegyzés neve leír. Egy új művelet törli a megismételhető állapotokat, és a szerkesztő legfeljebb kétszáz lépést őriz meg.

Egy felhasználói művelet néha sok egymás utáni módosításból áll. Egy szín csúszkával történő állítása vagy egy szám begépelése egy közös összevonási kulccsal érkezik, és a rövid időn belül egymást követő módosítások egyetlen bejegyzésbe olvadnak. A vásznon végzett húzás közben, például egy alakzat mozgatásakor, csak egy előnézet változik, és az előzményekbe csak a felengedéskor kerül egy lépés. Ha a húzás végül semmit sem változtatott, bejegyzés sem keletkezik.

3.7. Az eszközök közös felülete

Minden eszköz ugyanazt a felületet valósítja meg, amelyet a 3.1. programkód mutat. A vászon a böngésző mutatóeseményeit dokumentumkoordinátákká alakítja, közben figyelembe veszi a nagyítást, a belépett csoportot és az illesztést, majd az aktuális eszköznek adja tovább. A `target` mezőből az eszköz azt is megtudja, mi van a mutató alatt.

```
interface PointerInfo {
  p: Point;          // document coordinates, snapped
  raw: Point;        // document coordinates, never snapped
  shift: boolean; alt: boolean; ctrl: boolean;
  target: Element | null;   // the element under the pointer
}
interface Tool {
  down(i: PointerInfo): void;
  move(i: PointerInfo): void;
  up(i: PointerInfo): void;
  dblclick?(i: PointerInfo): void;
  key?(e: KeyEvent): boolean;  // true: the key was consumed
  escape(): boolean;           // false: the canvas deselects
  deactivate(): void;          // finish or discard pending work
  busy(): boolean;
}
```

3.1. Programkód. Az eszközök közös felülete (`tools/types.ts`, rövidítve)

Az eszközök nem érik el közvetlenül a DOM-ot. Rajzolás közben az előnézetet és a segédvonalakat egy ideiglenes store-ba írják, amelyet a vászon jelenít meg, a kész alakzatot pedig az `editor` műveleteivel adják a dokumentumhoz. Az eszközök nevét, ikonját és billentyűjét egyetlen katalógus tartalmazza, ebből dolgozik az eszköztár és a súgó is, ezért a kettő nem térhet el egymástól.

A toll és a körív eszköz egy közös, éppen rajzolt útvonalon dolgozik. Rajzolás közben a B és az A billentyűvel váltva ugyanaz az útvonal folytatódik, és egy alakzaton belül egyenes szakaszok, Bézier-görbék és körívek követhetik egymást. Ugyanez a közös útvonal teszi lehetővé, hogy egy kijelölt nyitott útvonal végpontjáról indított rajzolás azt az útvonalat folytassa.

3.8. Tárolás és fájlformátumok

A szerkesztő a rajzot, a beállításokat és az exportált fájlokat külön kezeli. A rajz projektfájlba menthető, amely egy formátumazonosítót, egy verziószámot és magát a dokumentumot tartalmazó JSON-dokumentum. Az SVG csak saját kiterjesztésekkel őrizhetné meg mindazt, amire a szerkesztőnek szüksége van, például az alakzatok zárolását, a segédvonalakat vagy a forgatási középpontokat. Az automatikus mentés ugyanezt a formátumot írja a böngésző helyi tárolójába.

A rács, az illesztés, a panelek mérete és a hasonló beállítások azt írják le, hogyan dolgozik a felhasználó, és nem azt, amit rajzolt. Ezek ezért a böngésző helyi tárolójába kerülnek, és egy projektfájl bárhol megnyitva ugyanúgy néz ki. Kivételt a segédvonalak jelentenek, mert azok egy adott rajzhoz tartoznak, ezért a projektfájlba kerülnek. Az exportált SVG szabványos, a szerkesztő saját adatai nem szerepelnek benne. A raszteres képek az exportált SVG-ből készülnek, amelyet a szerkesztő a böngésző vásznára rajzol.

A böngésző helyi tárolójának mérete eredetenként korlátozott [80, 53], egy nagy beágyazott képet tartalmazó rajz el is érheti ezt a korlátot. Ha az automatikus mentés nem sikerül, a szerkesztő figyelmeztet, és a projektfájlba mentés továbbra is elérhető.

4. fejezet

Megvalósítás

A fejezet a terv megvalósításának azokat a részeit mutatja be, ahol a megoldás nem magától értetődő. Ilyen a találatvizsgálat, az elforgatott alakzatok méretezése, a körívek és a szabadkézi vonalak előállítása, a logikai műveletek, a szöveg útvonallá alakítása, az SVG importálása és az érintéses használat. A programkódok rövidített részletek, a megjegyzéseik a forráskódhoz igazodva angol nyelvűek.

4.1. A fejlesztés menete

A szerkesztő több fejlesztési körben, fokozatosan készült. Egy kör egy funkciócsoport megvalósításával kezdődött, például a logikai műveletekkel, a betűtípusok kezelésével vagy az érintéses vezérléssel, és az új kódhoz automatikus tesztek is készültek. Ezután a felületet böngészőben végigpróbáltam, a talált hibákat és a használat közben felmerült módosítási igényeket listába gyűjtöttem, és ezek adták a következő kör feladatait. A kézi tesztelésből származó hibák regressziós tesztjeit az 5.5. szakasz mutatja be.

A szerkesztő a 3.2. táblázatban felsorolt könyvtárakra és a vele szállított, SIL Open Font License alatt terjesztett betűtípusokra épül. Minden más, így az adatmodell, az előzménykezelés, az eszközök, a geometriai számítások nagy része, az SVG-importáló és -exportáló, a betűtípusok kezelése, a felület komponensei és a tesztek a projekt keretében készültek.

A tervezési döntéseket és azok indoklását a forráskód mellett egy döntési napló, az alkalmazás felépítését egy architektúra-leírás rögzíti, mindkettő megtalálható a mellékelt adathordozón. A munka a dolgozat alapján megismételhető. A függőségek pontos verzióit a 3.2. táblázat tartalmazza, a telepítés, a futtatás és a tesztek indításának lépéseit pedig az adathordozó leírása sorolja fel.

4.2. A projekt felépítése

A forráskód az `src` mappában van. A belépési pont a `main.ts`, amely az `App.svelte` gyökérkomponenst jeleníti meg az oldalon, a többi kód pedig az `src/lib` almappáiban található, a 3.4. szakaszban leírt rétegek szerint. A mappák tartalmát és méretét a 4.1. táblázat foglalja össze.

A gyökérkomponenssel és a globális stíluslappal együtt a forráskód körülbelül 14 300 sor. A tesztek a `tests` mappában vannak, ezeket az 5. fejezet mutatja be.

4.1. táblázat. A forráskód mappái

Mappa	Tartalom	Fájl	Sor
`utils`	adatmodell, geometria, útvonalak, SVG-import és -export, logikai műveletek, vágólap	32	5183
`stores`	dokumentum és előzmények, felület, nézet, szerkesztési műveletek	11	1158
`tools`	a tizenhárom eszköz és közös segédfüggvényeik	18	1270
`fonts`	betűtípus-katalógus, Google Fonts, helyi és beépített betűtípusok	17	1429
`components`	újrahasználható komponensek, például a színválasztó, az objektumlista és a párbeszédablakok	20	2850
`layouts`	a felület nagy egységei: vászon, menü, parancssáv, panelek	6	2060

4.3. Megjelenítés és találatvizsgálat

A vászon maga is egy SVG-elem, amelyben a `ShapeRenderer` komponens rajzolja ki az alakzatokat. A komponens ugyanazokat az attribútumokat írja ki, mint az exportáló, így a képernyőn látott kép megegyezik a mentett fájllal. Az értékeket mindig kiírja, akkor is, ha megegyeznek az SVG alapértékeivel, mert a szerkesztő alapértékei eltérnek azoktól. Az SVG alapértelmezett vonalvége például a levágott `butt`, a sarka a hegyes `miter` [76], a szerkesztőben viszont mindkettő lekerekített.

Minden kattintásnál el kell dönteni, mi van a mutató alatt: egy alakzat, egy fogantyú, egy csomópont vagy az üres vászon. Ezt a feladatot találatvizsgálatnak, angolul *hit-testing*nek nevezik. Elforgatott alakzatokra, vastag körvonalakra, görbékre és szövegekre pontosan kiszámolni sok geometriai kódot igényelne. A böngésző viszont ezt a számítást már elvégzi, mégpedig pontosan úgy, ahogyan az elemeket kirajzolja, és a szerkesztő rá bízza a feladatot. Minden kirajzolt elem egy `data-` kezdetű attribútummal jelzi, mit ábrázol, egy alakzat például `data-shape-id`, egy méretező fogantyú `data-handle` attribútummal. Az esemény célelemétől a szülők felé haladva az első ilyen attribútum megmondja, mire kattintott a felhasználó.

Két esetben a böngésző viselkedését ki kellett egészíteni. Egy vékony vonalat vagy egy kitöltés nélküli útvonalat nehéz eltalálni, mert csak a körvonala kattintható. Az ilyen alakzatok alá a szerkesztő egy láthatatlan másolatot rajzol, amelynek körvonala a képernyőn legalább tíz képpont széles, és a CSS `pointer-events` tulajdonsága szerint csak ez a körvonal fogadja a mutató eseményeit [39]. A zárolt alakzatok `pointer-events: none` beállítást kapnak, és a kattintások átmennek rajtuk az alattuk lévő elemekre. Egy zárolt háttér így sem kijelöléssel, sem területkijelöléssel, sem vágással, sem a foltecsettel nem módosulhat véletlenül.

4.4. Nézet, nagyítás és illesztés

A rajzterületnek nincsenek görgetősávjai és szélei. A nézetet egy eltolás és egy nagyítási arány írja le, az eltolás pedig a tervezőprogramokhoz hasonlóan tetszőleges lehet. Görgetősávokkal a nézet csak a lap és egy margó területén mozoghatna, a szerkesztő viszont a lapon kívüli alakzatokat is megengedi. A nagyítás 5% és 2000% között állítható.

Nagyításkor a kurzor alatti dokumentumpontnak a helyén kell maradnia. Legyen s a nézet eltolása, a a kurzor távolsága a rajzterület szélétől, p a lap előtti állandó margó, z_0 a régi és z_1 az új nagyítás. A kurzor alatti dokumentumkoordináta $(s + a - p)/z_0$, és ennek az új nagyítás mellett is a távolságra kell lennie. Ebből az új eltolás:

$$s' = \frac{s + a - p}{z_0} z_1 + p - a.$$

Ugyanezt a képletet használja a görgő, a két ujjas nagyítás és a nagyító gombok is, az utóbbiak a rajzterület középpontjával.

Az illesztés a mozgatott vagy rajzolt alakzat széleit és középvonalait veti össze a többi alakzat, a lap és a segédvonalak megfelelő koordinátáival. Egy koordináta akkor illeszkedik, ha legfeljebb hat képernyőpontnyira van a céltól. A küszöb a képernyőn állandó, dokumentumegységben a nagyítással osztva kell számolni. Az illesztést jelző vonalak vékony világos szegélyen futnak, így egy rózsaszín jelzővonal egy rózsaszín alakzaton is látható marad.

4.5. Az előzmények megvalósítása

A 3.6. szakaszban leírt idővonalat a `documentStore` valósítja meg. Központi függvénye a `commit`, amelyet a 4.1. programkód mutat. A függvény egy átalakítást kap, amely a régi dokumentumból újat készít. Ha az átalakítás ugyanazt az objektumot adja vissza, a művelet semmit sem változtatott, és nem keletkezik bejegyzés. Ha a hívás összevonási kulcsot is megad, és az előző lépés egy másodpercen belül ugyanezzel a kulccsal történt, az új állapot a legutóbbi bejegyzés helyére kerül.

```
commit(label: string, fn: (d: DocumentState) => DocumentState,
       opts?: { coalesce?: string }) {
  core.update(c0 => {
    const c = cancelled(c0);              // drop an unfinished preview
    const next = fn(c.present.doc);
    if (next === c.present.doc) return c; // nothing changed: no entry
    const now = Date.now(), key = opts?.coalesce ?? null;
    if (key && key === c.lastKey && now - c.lastTime < COALESCE_MS)
      return { ...c, present: { doc: next, label: c.present.label },
               future: [], lastTime: now };          // merge
    return { past: trim([...c.past, c.present]),     // at most 200
             present: { doc: next, label }, future: [],
             base: null, lastKey: key, lastTime: now };
  });
}
```

4.1. Programkód. Egy visszavonható lépés rögzítése (`stores/document.ts`, rövidítve)

A vásznon történő húzáshoz a tároló előnézeti módot kínál. A `beginPreview` megjegyzi a húzás előtti állapotot, ezután a `preview` hívások bejegyzés nélkül módosítják a jelenlegi állapotot, a `commitPreview` pedig egyetlen lépésként rögzíti a húzás eredményét. A `cancelPreview` visszaállítja a megjegyzett állapotot, ezt hívja például az Esc billentyű húzás közben. Az előzmények panelen egy bejegyzésre kattintva a `goTo` függvény az idővonalat a kiválasztott pontnál osztja ketté, és onnan a visszavonás és a megismétlés ugyanúgy folytatható.

Mivel a modell függvényei a változatlan részekre a régi hivatkozást tartják meg, két dokumentumállapot összehasonlítása egyetlen hivatkozás-összehasonlítás. Erre épül a

fenti üres műveletek kiszűrése, és ugyanígy dönti el a szerkesztő azt is, hogy van-e mentetlen változás.

4.6. Rajzoló eszközök

A rajzoló eszközök közül a téglalap, az ellipszis, a vonal és a sokszög egyszerű húzásokból és kattintásokból állítja elő az alakzatot. A toll, a körív és a szabadkézi rajz megvalósítása több számítást igényel, ezért ez a szakasz ezekkel foglalkozik.

4.6.1. Toll és körív

A toll a tervezőprogramokban megszokott módon működik. Egy kattintás sarokpontot helyez el, egy húzás sima csomópontot, amelynek fogantyúit a húzás iránya és hossza adja meg. Az útvonalat dupla kattintás, az Enter vagy az Esc billentyű fejezi be, az első pontra kattintás pedig bezárja.

A körív eszköz három ponttal dolgozik, a két végponttal és egy harmadikkal, amelyen az ívnek át kell haladnia. Ez a mozdulat egérrel és ujjal is ugyanúgy végrehajtható. A három pont meghatározza a köréjük írható kört. Ha a pontokat a harmadik ponthoz, P-hez viszonyítva $\mathbf{a} = A - P$ és $\mathbf{b} = B - P$ jelöli, a kör középpontja

$$C = P + \frac{1}{d}\Big(b_y|\mathbf{a}|^2 - a_y|\mathbf{b}|^2,\; a_x|\mathbf{b}|^2 - b_x|\mathbf{a}|^2\Big), \qquad d = 2\,(a_x b_y - a_y b_x).$$

Ha d közel nulla, a három pont egy egyenesre esik, és az eszköz egyenes szakaszt rajzol. A Shift billentyű lenyomásakor az ív középponti szöge tizenöt fokos lépésekben változik, így negyed- és félkör is pontosan beállítható.

A modell csak harmadfokú Bézier-görbéket ismer, ezért a körívet közelíteni kell. Egy φ középponti szögű, r sugarú ív egyetlen harmadfokú görbével akkor közelíthető jól, ha a két fogantyú az ív érintőjének irányába mutat, és a hosszuk

$$h = \frac{4}{3}\tan\frac{\varphi}{4}\,r.$$

Negyedkörre ez a sugár körülbelül 0,5523-szorosa, a közelítés legnagyobb eltérése pedig a sugár 0,027%-a [23, 56]. Nagyobb szögeknél a hiba gyorsan nő, ezért az eszköz az ívet legfeljebb 90 fokos darabokra bontja, ahogyan a 4.2. programkód mutatja.

```
export function arcNodes(arc: Arc): PathNode[] {
  const k = Math.max(1, Math.ceil(Math.abs(arc.sweep) / (Math.PI / 2) - 1e-9));
  const phi = arc.sweep / k;                          // at most 90 degrees
  const h = (4 / 3) * Math.tan(phi / 4) * arc.r;      // handle length
  const nodes: PathNode[] = [];
  for (let i = 0; i <= k; i++) {
    const ang = arc.start + phi * i, p = at(ang), tg = tangent(ang);
    nodes.push({ x: p.x, y: p.y,
      hin:  i > 0 ? { x: p.x - tg.x * h, y: p.y - tg.y * h } : null,
      hout: i < k ? { x: p.x + tg.x * h, y: p.y + tg.y * h } : null,
      smooth: i > 0 && i < k });
  }
  return nodes;
}
```

4.2. Programkód. Körív felbontása Bézier-csomópontokra (`utils/arc.ts`, rövidítve)

A toll és a körív egy közös, éppen rajzolt útvonalon dolgozik, amelyet a `pathDraft` modul tárol. Minden új ív ott kezdődik, ahol az előző szakasz véget ért, így a körív eszköz a tollhoz hasonlóan használható. A szakaszok egymást követik, a rajzolást dupla kattintás vagy az Enter billentyű zárja le. Ha a rajzolás egy kijelölt nyitott útvonal végpontján kezdődik, a közös vázlat ezt az útvonalat veszi át. Az új szakaszok hozzá kerülnek, az alakzat megtartja azonosítóját, nevét és stílusát, a másik végpontjára kattintva pedig bezárul. Ugyanezt a végpontkeresést használja a szabadkézi eszköz is.

4.6.2. Szabadkézi rajz

Az egér vagy az ujj mozgása sok mintát ad, amelyekből közvetlenül egy töröttvonal is készülhetne. Ez azonban sok száz csomópontot jelentene, és a kéz remegését is megőrizné, ezért az eszköz két lépésben dolgozza fel a mintákat.

Az első lépés a Ramer-Douglas-Peucker algoritmus [61, 7]. Az algoritmus a két végpontot összekötő szakasztól legtávolabbi mintát keresi. Ha ez a távolság egy tűréshatárnál nagyobb, a pontot megtartja, és a két keletkező részre megismétli a keresést, különben a köztes pontokat elhagyja. A megvalósítás rekurzió helyett saját vermet használ, hogy egy hosszú vonal se tölthesse meg a hívási vermet. A tűréshatár a képernyőn nagyjából másfél képpont, így erősebb nagyításnál a vonal részletesebb marad.

A második lépés a megmaradt pontokból sima görbét készít. A P_i csomópont fogantyúi a szomszédos pontokat összekötő irányba mutatnak, a Catmull-Rom spline-ok mintájára [5]:

$$\mathbf{t}_i = \frac{P_{i+1} - P_{i-1}}{6}, \qquad H_i^{\text{be}} = P_i - \mathbf{t}_i, \qquad H_i^{\text{ki}} = P_i + \mathbf{t}_i.$$

Az eredmény néhány tucat sima csomópont, amely a csomóponteszközzel kényelmesen tovább alakítható.

4.7. Alakzatok szerkesztése

A már megrajzolt alakzatokat a kijelölő eszköz mozgatja, méretezi és forgatja, a csomóponteszköz pedig az alakjukat módosítja. Mindkettőnél az okozza a nehézséget, hogy az alakzatok el lehetnek forgatva.

4.7.1. Elforgatott alakzatok méretezése

Egy elforgatott alakzat méretezésekor a megfogott fogantyúval szemközti saroknak vagy élnek a helyén kell maradnia. Az alakzat forgatása a határoló téglalap közepe körül értendő, a méretezés viszont elmozdítja ezt a középpontot, és egy egyszerű méretezés után az alakzat elcsúszna. A `resizeShape` függvény ezért három lépésben dolgozik.

1. A mutató helyét a forgatás középpontja körül $-\theta$ szöggel visszaforgatja, így a mutató az alakzat saját, el nem forgatott koordináta-rendszerébe kerül.

2. Ebben a rendszerben a megfogott fogantyú szerint új határoló téglalapot számol, Shift billentyűvel az arányokat megtartva, és az alakzatot ebbe a téglalapba méretezi.

3. A rögzített pont helyét a régi és az új téglalapban is kiszámolja, mindkettőt a saját középpontja körül θ szöggel elforgatva. A két pont különbségével eltolja az alakzatot, és a rögzített pont a képernyőn ugyanott marad.

Több kijelölt alakzat a közös, tengelyekkel párhuzamos határoló téglalap szerint méreteződik. Minden alakzat középpontja a téglalappal együtt mozog, a mérete ugyanolyan arányban változik, a forgatása pedig megmarad. Ez eltolásra és arányos méretezésre pontos, egy elforgatott alakzat nem arányos méretezése viszont csak közelítés, mert a modell nem ismeri a nyírást. Ugyanez a szabály érvényes a csoportok tagjaira is.

4.7.2. A forgatási középpont

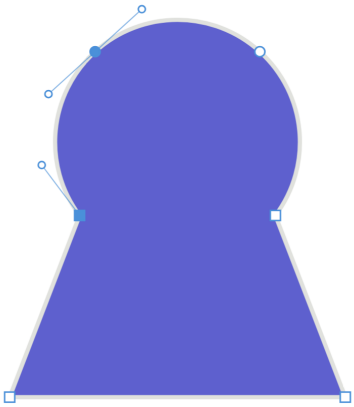
A 3.5. szakasz szerint az áthelyezett forgatási középpont csak a forgatás végrehajtását befolyásolja. Ha egy alakzatot Δ szöggel egy tetszőleges O pont körül kell elforgatni, a szerkesztő a tárolt szöghöz hozzáadja Δ-t, a téglalap C középpontját az O pont körül elforgatja, és az alakzatot az új középpontba tolja. Az eredmény ugyanaz, mintha az alakzat valóban az O pont körül fordult volna el, az exportált fájlban azonban továbbra is egyetlen `rotate` transzformáció szerepel. A geometria panel is ezt a műveletet végzi, amikor a felhasználó számmal adja meg a szöget.

4.7.3. Csomópontok szerkesztése

A csomóponteszköz, a felületen *Edit Nodes*, minden alakzattípushoz szerkeszthető pontokat kínál. Útvonalnál ezek a csomópontok és a fogantyúk, téglalapnál a saroksugarak, ellipszisnél a két sugár, vonalnál a végpontok. A pontok az alakzat saját, el nem forgatott koordináta-rendszerében értendők, és ugyanaz a kód kezeli az elforgatott és a nem elforgatott alakzatokat.

Több csomópont is kijelölhető Shift-kattintással vagy egy téglalap húzásával, akár az alakzat belsejéből indítva. A kijelölt csomópontok együtt mozognak, a nyílbillentyűkkel léptethetők, együtt törölhetők, és mindegyik fogantyúja látszik. Dupla kattintás az útvonalon új csomópontot illeszt be, egy csomóponton pedig a sarok és a sima típus között vált. A 4.1. ábra egy kulcslyuk alakú útvonalat mutat két kijelölt csomóponttal.

A fogantyúk abszolút koordinátákként tárolódnak, nem a csomóponthoz viszonyított eltolásként. Az eltolás, a méretezés és a tükrözés emiatt minden pontra ugyanaz a leképezés, a csomópontokat és a fogantyúkat nem kell külön kezelni. Sima csomópontnál az egyik fogantyú mozgatása a másikat az ellenkező irányba igazítja, és a görbe törés nélkül halad át a csomóponton.

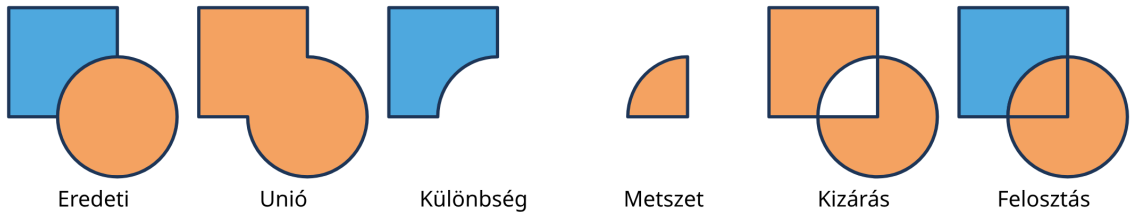


4.1. ábra. Két kijelölt csomópont a fogantyúival

4.8. Logikai műveletek és vágás

A logikai műveletek a paper.js könyvtár `unite`, `subtract`, `intersect` és `exclude` metódusaira épülnek [32]. A szerkesztő a kijelölt alakzatokat előbb paper.js útvonalakká alakítja. A téglalapok, ellipszisek és vonalak ehhez útvonallá válnak, a forgatás beépül a csomópontok koordinátáiba, a fogantyúk pedig a csomóponthoz viszonyított értékekre alakulnak, mert a paper.js ezt a formát várja. Az eredményt ugyanígy alakítja vissza.

A műveletek a halmozási sorrendet követik. Az unió, a metszet és a kizárás eredménye a legfelső alakzat stílusát kapja. A különbség a legalsó alakzatból vágja ki az összes többit, és a legalsó stílusát tartja meg. A felosztás minden alakzatot a körvonalaik által kijelölt darabokra bontja, és minden darab annak az alakzatnak a stílusát kapja, amelyik legfelül fedi. A paper.js dokumentációja szerint a felosztás egy különbség és egy metszet együttese [32]. Több alakzatra a szerkesztő ezt lépésenként terjeszti ki. Minden következő alakzat az eddigi darabokat egy rajta kívüli és egy vele közös részre vágja, saját magából pedig az eddig le nem fedett rész lesz új darab. A felületen a műveletek *Unite*, *Subtract*, *Intersect*, *Exclude* és *Divide* néven érhetők el. A 4.2. ábra két egymást fedő alakzaton mutatja az öt műveletet.



4.2. ábra. Két alakzat és a rajtuk végzett öt logikai művelet eredménye

Az olló eszköz kétféleképpen vág. Egy kattintás az útvonal körvonalán csak abban a pontban vágja el az útvonalat: egy nyitott útvonal két részre esik, egy zárt útvonal pedig ott kinyílik. A vágás helyére új csomópont kerül, amelyet a de Casteljau-algoritmus számol ki [10], így az alakzat alakja nem változik. Egy húzás ezzel szemben kést rajzol, amely mentén minden eltalált alakzat kettéválik. Zárt alakzatnál a darabok is zártak,

vagyis a körvonal a vágás mentén is végigfut.

Egy vonalnak nincs területe, ezért közvetlenül nem lehet logikai művelet operandusa. A szerkesztő a késből egy területet épít, ahogyan a 4.3. programkód mutatja. A kés két végét az érintő irányában messzire meghosszabbítja, és a vonalat az egyik oldalán körbezárja. A darabok ezután egy metszet és egy különbség eredményeként állnak elő. Ha a felhasználó a kést a kiindulópontjáig visszahúzza, az lasszóként működik, és maga a zárt vonal lesz a vágó terület.

```
export function knifeRegion(paper: Paper, knife: Item, far: number): Item {
  const first = knife.firstSegment.point, last = knife.lastSegment.point;
  const t0 = knife.getTangentAt(0).normalize();
  const t1 = knife.getTangentAt(knife.length).normalize();
  const start = first.subtract(t0.multiply(far)), end = last.add(t1.multiply(far));
  const dir = end.subtract(start).normalize();
  const side = new paper.Point(-dir.y, dir.x).multiply(far * 2);
  const region = knife.clone({ insert: false });
  region.insert(0, start);
  region.add(end, end.add(side), start.add(side));
  region.closed = true;
  return region;
}
// the two pieces:  target.intersect(region)  and  target.subtract(region)
```

4.3. Programkód. A késből épített vágó terület (`utils/boolean.ts`)

4.9. Körvonalak

A körvonal beállításai közül a szaggatás és a körvonal útvonallá alakítása igényelt saját megoldást. Ez a szakasz ezeket és a velük rokon foltecsetet tárgyalja.

4.9.1. Szaggatott vonalak

Az SVG `stroke-dasharray` tulajdonsága abszolút hosszakat ír le [76]. Egy `6 4` minta vékony vonalon szaggatottnak látszik, egy 12 egység vastag, lekerekített végű vonalon viszont már folytonosnak, mert a vonalvégek kitöltik a hézagokat. A szerkesztő ezért a mintákat a vonalvastagsághoz viszonyítva kezeli. A szaggatott minta szakaszai a vastagság háromszorosai, a hézagai a kétszeresei, a pontozott minta pontjai pedig pontosan olyan hosszúak, amilyen vastag a vonal.

A lekerekített és a négyzetes vonalvég mindkét irányban a vastagság felével túlnyúlik a szakaszokon [76]. A szerkesztő ezt úgy egyenlíti ki, hogy a szakaszok hosszából levonja a vastagságot, a hézagokhoz hozzáadja, és a látható minta minden vonalvégnél ugyanolyan lesz. A pont ennek megfelelően egy majdnem nulla hosszú szakasz, amelyet a vonalvég tesz láthatóvá. Ha a felhasználó a vastagságot vagy a vonalvéget módosítja, a minta újraszámolódik. A kézzel beírt vagy importált minták viszont az SVG-hez hasonlóan abszolút értékek maradnak. Az exportált fájlban minden minta szabványos `stroke-dasharray` érték, amelyet bármely megjelenítő ugyanúgy rajzol ki.

4.9.2. Körvonal útvonallá alakítása és a foltecset

A körvonal útvonallá alakítása a kirajzolt körvonal területét kitöltött alakzattá teszi. Ehhez a görbét mindkét oldalán a vastagság felével el kell tolni, egy Bézier-görbe el-

toltja azonban általában már nem Bézier-görbe, ezért csak közelíthető [9]. A szerkesztő az útvonalat sűrűn, körülbelül másfél képpontonként mintavételezi, majd a Clipper könyvtár sokszögeltolásával a vonal mindkét oldalára kiterjeszti a vastagság felével, a vonalvég és a sarok beállítását is figyelembe véve [30]. Szaggatott vonalnál minden szakasz külön sokszög lesz. Az eredmény sokszögközelítés, amelyben az átfedések és a lyukak helyesen oldódnak fel. Egy apró eltérés azért marad, mert a hegyes sarkok korlátját túllépő csúcsokat a Clipper derékszögben vágja le, az SVG pedig ferdén.

A foltecset ugyanezt a technikát használja festésre. A húzás nyomvonalát az ecset méretére terjeszti ki, majd egyesíti azokkal a zárt alakzatokkal, amelyeket érint, és amelyeknek ugyanaz a kitöltése és körvonala. Az egyesített körvonalat a Ramer-Douglas-Peucker egyszerűsítés és a Catmull-Rom fogantyúk néhány sima csomóponttá alakítják, a hegyes sarkok azonban sarokpontok maradnak.

4.10. Szövegek és betűtípusok

A szöveg a szerkesztő legösszetettebb alakzata, mert a megjelenése a betűtípustól függ, amely a felhasználó gépén, a Google Fonts gyűjteményében vagy a szerkesztővel együtt szállítva is elérhető lehet. Az alábbiakban sorra veszem a szöveg szerkesztését, a betűtípusok forrásait és a szöveg útvonallá alakítását.

4.10.1. Szövegszerkesztés és mérés

A szöveg közvetlenül a vásznon szerkeszthető. Szerkesztés közben a szerkesztő egy `<textarea>` elemet helyez el egy `<foreignObject>` elemben, pontosan ott, ahol a szöveg megjelenik. Az Esc vagy a Ctrl+Enter billentyű, illetve egy máshová történő kattintás befejezi a szerkesztést.

A szöveg határoló téglalapjához ismerni kell a sorok szélességét. Ezt a böngésző vásznának `measureText` metódusa adja meg ugyanazzal a betűtípussal, amellyel a szöveg megjelenik, a betű- és szóközöket hozzászámolva. Egy webes betűtípus betöltése után a mérés eredménye megváltozhat, ezért a szerkesztő minden betöltött betűtípus után növel egy számlálót. A mérést végző reaktív kifejezések ettől a számlálótól függenek, és a kijelölő keret mindig a valódi szöveghez igazodik.

4.10.2. Betűtípusok forrásai

A betűtípusok három forrásból származhatnak. Az első a Google Fonts teljes gyűjteménye, amelyet a szerkesztő a CSS2 API-n keresztül ér el. Az API `text` paramétere esetén csak a felsorolt karaktereket tartalmazó, akár 90%-kal kisebb fájlt ad vissza [24]. A betűtípus-választóban ezért minden sor csak a betűtípus nevének betűit kéri le, és a lista görgetése néhány száz bájtos kérésekkel jár. Ha a kért stílus nem létezik a családban, az API az egész kérést elutasítja [24]. A szerkesztő ezért a Fontsource metaadatai alapján csak a ténylegesen létező stílusokat kéri le [21], és ha mégis hiba történik, egy rövid visszalépési láncot követ, például félkövér dőltről félkövérre, majd normál dőltre.

A második forrás a felhasználó gépe. A Local Font Access API a felhasználó engedélyével felsorolja a telepített betűtípusokat, és hozzáférést ad a fájljaikhoz [38]. Ezt az API-t csak a Chromium-alapú böngészők valósítják meg. Más böngészőkben a szerkesztő néhány gyakori betűtípust a vásznon mért szélesség alapján ismer fel. Ezek megjeleníthetők, de útvonallá nem alakíthatók.

A harmadik forrás a szerkesztővel együtt szállított betűtípusok köre. A `serif`, a `sans-serif` és a többi általános CSS betűcsalád mögött minden böngészőben más betűtípus áll. A szerkesztő ezekhez nyílt licencű betűtípusokat tartalmaz [62], és ezekkel jeleníti meg őket. A képernyőn látott szöveg és az útvonallá alakított változat így ugyanabból a betűtípusból készül. A fájlok írásrendszerenként külön részekre bontva töltődnek be, csak akkor, ha a szöveg tartalmazza az adott karaktereket.

4.10.3. Szöveg útvonallá alakítása

Az útvonallá alakításhoz a betűket ugyanúgy kell elrendezni, ahogyan a böngésző, vagyis azonos sorközzel, igazítással, betű- és szóközzel, alávágással és ligatúrákkal. Erre a fontkit könyvtár szolgál, amely teljes OpenType-szövegszedést végez [20]. Az előbb kipróbált opentype.js könyvtár egyes betűtípusok alávágási tábláit nem tudta beolvasni, így a betűpárok távolsága eltért a böngészőben látottól. Egy Inter betűtípussal szedett mintaszövegen a képpontok mintegy 40%-a eltért, ezért esett a választás a fontkit könyvtárra, amelynek pontosságát a 6.2.2. alszakasz méri. Változó betűtípusoknál a szerkesztő a vastagság és az optikai méret tengelyét a szöveg beállításaira állítja. Ha a kért vastagság vagy dőlés nincs meg a betűtípusban, a böngészőkhöz hasonlóan mesterségesen állítja elő.

Az útvonallá alakításhoz szükség van a betűtípus fájljára. Google Fonts-betűtípusnál ez egy `text` paraméteres kérés, amely pontosan a szövegben szereplő karaktereket tartalmazó, kis méretű WOFF2-fájlt ad [24, 78], helyi betűtípusnál pedig a Local Font Access API adja a fájlt. Ha a betűtípus egyik forrásból sem érhető el, a szerkesztő nem helyettesíti egy hasonlóval, hanem az okát megadva elutasítja az átalakítást. Egy csendes helyettesítés a felhasználó tudta nélkül változtatná meg a rajzot. A vegyesen jobbról balra és balról jobbra haladó szövegek kétirányú átrendezése nincs megvalósítva, ilyenkor az átalakítás figyelmeztet.

4.11. SVG-importálás és -exportálás

Az importálás egy idegen, akár hibás vagy rosszindulatú fájlt alakít a szerkesztő modelljére, az exportálás pedig a modellből szabványos fájlt készít. Az importálás dönti el, milyen fájlok és milyen korlátozásokkal szerkeszthetők, ezért ezt tárgyalom részletesebben.

4.11.1. Importálás

Az importálás a böngésző `DOMParser` objektumával kezdődik, amely az SVG-szöveget XML-dokumentummá alakítja. Ha az elemzés hibát jelez, vagy a gyökérelem nem `svg`, a szerkesztő elutasítja a fájlt. Az importáló ezután bejárja az elemfát, és minden támogatott elemből a szerkesztő saját alakzatát állítja elő. Közben nyilvántartja az egymásba ágyazott transzformációkat és az öröklődő stílusokat, mind a megjelenési attribútumokból, mind a `style` attribútumból. A 4.2. táblázat összefoglalja, mit támogat az importáló, és mit nem.

A táblázat jobb oldali oszlopában szereplő szerkezetek közül egyik sem tűnik el szó nélkül. Minden kihagyott vagy közelített szerkezetről érthető üzenet szól, amelyet a szerkesztő az importálás után megjelenít, a felhasználó így pontosan tudja, mi változott a fájlon. Hogy ez valós fájlokon mennyire teljesül, azt a 6.2.1. alszakasz méri. A `script`

4.2. táblázat. Az SVG-importáló által támogatott és nem támogatott szerkezetek

Támogatott	**Nem támogatott vagy közelített**
`svg`, `g`, `a`, `use` a `defs` és `symbol` elemekkel; egymásba ágyazott transzformációk tetszőleges mátrixszal	minták, amelyek szürke kitöltést kapnak
`switch`, amelyből az első olyan ág kerül át, amelynek feltételei, például a `systemLanguage`, teljesülnek	`<style>` stíluslapok szabályai
`rect`, `circle`, `ellipse`, `line`, `polyline`, `polygon`, `path` minden paranccsal, `text` és `tspan`	vágóútvonal, maszk, szűrő, jelölők (például nyílhegyek), keverési módok
megjelenési attribútumok és `style` attribútum öröklődéssel és az SVG alapértékeivel; `fill-opacity`, `stroke-opacity`	útvonalra illesztett szöveg (`textPath`), `tref`, SVG-betűtípusok, animációk
`viewBox` méretezés; csoportok átlátszósága és forgatása	nyírt képek és szövegek, amelyeket csak forgatás és méretezés közelít
lineáris és sugárirányú színátmenetek, `gradientTransform`, `href` láncok	színátmenetek fókuszpontja és ismétlési módja
kitöltött összetett útvonalak nyitott és zárt részútvonalakkal	körvonalas útvonal vegyes részútvonalai, amelyek külön alakzatokra válnak
`<image>` `data:` vagy `http(s)` címmel	külön fájlra hivatkozó képek

és a `foreignObject` elem soha nem kerül át a szerkesztőbe, így egy importált fájl nem futtathat kódot. A bejárás legfeljebb negyven szint mélyre halad, és legfeljebb ötezer alakzatot hoz létre, egy szándékosan túlméretezett fájl így sem teheti használhatatlanná az oldalt. Ha a korlát miatt alakzatok maradnak ki, erről is figyelmeztetés szól.

A téglalapok, ellipszisek, vonalak és szövegek megtartják a típusukat, ha a rájuk ható transzformáció csak eltolás, méretezés és forgatás. Nyírás vagy tükrözés esetén útvonallá alakulnak, mert a modell ezeket csak útvonalként tudja pontosan ábrázolni. Az útvonalak elliptikus íveit az importáló az SVG implementációs függeléke szerint előbb középponti paraméterezésre alakítja [74], majd a körívekhez hasonlóan legfeljebb 90 fokos harmadfokú szakaszokkal közelíti.

4.11.2. Exportálás

Az exportált SVG szabványos dokumentum. A színátmenetek egy közös `<defs>` elembe kerülnek, az alakzatok pedig a 3.3. táblázatban felsorolt elemekként jelennek meg. A koordináták két tizedesjegyre kerekítve íródnak ki, ami a képernyőn nem látható eltérést okoz, de jelentősen csökkenti a fájl méretét.

A szöveg háromféleképpen hagyhatja el a szerkesztőt. Maradhat egyszerű `<text>` elem, ekkor a megjelenítő gépen is telepítve kell lennie a betűtípusnak. A Google Fonts-betűtípusok `data:` URL formában be is ágyazhatók a fájlba [37], mert egy URL-lel hivatkozott betűtípus nem töltődik be, ha az SVG képként vagy hálózat nélkül nyílik meg. Végül a szöveg útvonallá is alakítható, és akkor a fájlhoz semmilyen betűtípus nem kell. A helyi betűtípusok beágyazása külön választható, és figyelmeztetéssel jár, mert az ilyen betűtípusok licence gyakran tiltja a terjesztést.

A raszteres kép a dokumentum SVG változatából készül, amelyet a szerkesztő kép-

ként betölt, egy vászonra rajzol, majd PNG, JPG vagy WebP formátumban ment. A raszteres kép alapértelmezés szerint legalább 4000 képpont széles, hogy egy kis méretű lapról készült kép is éles legyen nagyobb kijelzőn vagy nyomtatásban. Az exportálás beállításaiban helyette méretarány vagy pontos szélesség is megadható. A vászon legnagyobb mérete böngészőnként és eszközönként eltér [35], a szerkesztő ezért oldalanként legfeljebb 16 384, összesen legfeljebb 268 millió képpontos képet készít, és ennél nagyobb méret kérésekor hibaüzenetet ad. Rajzolás előtt a szerkesztő a szövegeket útvonallá alakítja, hogy a kép ne függjön attól, hogy a képként betöltött SVG-ben a betűtípusok betöltődnek-e. Ha egy szöveg betűtípusa nem érhető el, például hálózat nélkül, a szöveg a böngésző betűtípusaival készül, és erről figyelmeztetés szól. Az SVG eleve a célméretben készül, a böngésző tehát a vektoros rajzot teljes felbontásban rajzolja ki, és nem egy kis képet nagyít fel.

4.12. Vágólap és tárolás

A másolás egyszerre két helyre ír. A rendszer vágólapjára SVG-szövegként kerül a kijelölés, így más programokba, például az Inkscape-be vagy egy szövegszerkesztőbe is beilleszthető [36]. Emellett a szerkesztő a kijelölt alakzatokat a böngésző helyi tárolójába is elmenti, amelyet ugyanannak az oldalnak minden lapja elér, az SVG-szöveget pedig egy erre a bejegyzésre hivatkozó XML megjegyzéssel jelöli meg. Beillesztéskor a következő sorrendben dönt:

1. a saját jelölésével ellátott SVG-szöveg esetén a tárolt alakzatok kerülnek vissza minden adatukkal, akár egy másik lapról is;
2. más programból származó SVG-szöveget az importálón keresztül, szerkeszthető alakzatokként illeszt be;
3. a vágólapon lévő kép képként kerül a rajzba;
4. ha a rendszer vágólapján nincs használható tartalom, a szerkesztő legutóbbi saját másolatát illeszti be.

A projektfájl betöltésekor a szerkesztő minden alakzatot mezőnként ellenőriz. Az ismeretlen típusú vagy hiányos alakzatokat eldobja, a számokat, például a betűméretet, a megengedett tartományra szorítja, az ismétlődő azonosítókat lecseréli, és a csoportok mélységét is korlátozza. Egy kézzel szerkesztett vagy sérült fájl így nem okozhat hibát a szerkesztő többi részében. Az automatikus mentés a dokumentum minden változása után 800 ezredmásodperc szünettel írja a projektet a böngésző helyi tárolójába [80]. A betöltéskori állapotot nem menti, nehogy egy üres dokumentum felülírjon egy visszaállított munkát. Ha a tároló megtelt, a szerkesztő figyelmeztet, hogy a munkát fájlba kell menteni.

4.13. Reszponzív felület és érintés

A keskeny elrendezést egy `(max-width: 640px)` médialekérdezés kapcsolja be [52]. Ugyanez a lekérdezés egy Svelte store-ban is elérhető, így a komponensek a viselkedésüket is hozzáigazíthatják, nem csak a stílusukat. Keskeny kijelzőn a tulajdonságpanel

alulról nyíló lappá válik, a parancssávból pedig néhány ritkábban használt vezérlő, például az átlátszóság kimarad, amely a panelen továbbra is elérhető.

A szerkesztő az egér, a toll és az ujj eseményeit egységesen, a Pointer Events szabvány eseményeivel kezeli [71]. A vásznon a böngésző saját érintéses görgetését és nagyítását a `touch-action` CSS-tulajdonság kapcsolja ki [49], ezeket a mozdulatokat a szerkesztő maga dolgozza fel. Három helyen kellett külön megoldást keresni.

Az első a két ujjas nagyítás. Egy érintés nem jut el azonnal az aktuális eszközhöz, hanem 90 ezredmásodpercig vár. Ha ezalatt egy második ujj is a képernyőhöz ér, a mozdulat nagyítás, és az eszköz egyáltalán nem értesül az első ujjról. E várakozás nélkül a toll minden nagyítás előtt egy felesleges csomópontot helyezne el. Nagyítás közben az ujjak közötti pont a helyén marad, és ha a két ujj együtt mozog, a nézet követi őket.

A második a helyi menü. Az iOS Safari böngésző nem küldi el a `contextmenu` eseményt, ezért a szerkesztő saját hosszú érintést valósít meg. Ha az ujj 550 ezredmásodpercig egy helyben marad, megnyílik a helyi menü.

A harmadik a fogantyúk mérete. Ha a fő mutatóeszköz az ujj, a fogantyúk és a csomópontok egy láthatatlan, 14 képpont sugarú, vagyis 28 képpont átmérőjű érzékeny területet kapnak. A WCAG 2.2 irányelv 2.5.8-as sikerkritériuma legalább 24×24 CSS-képpontos célméretet ír elő [77], ennek a szerkesztő fogantyúi megfelelnek. Parhi és társai vizsgálatában a hüvelykujjal használt telefonon a 9,2-9,6 mm-es célok voltak megbízhatóan eltalálhatók [59]. A 28 képpont a CSS 96 dpi-s névleges felbontásán körülbelül 7,4 mm, a fogantyúk tehát ennél kisebbek, cserébe sűrűn elhelyezkedő csomópontok mellett is külön-külön megfoghatók maradnak.

Hasonló kérdést vetett fel az objektumok panel. Az elemek ott húzással rendezhetők, érintőképernyőn azonban ugyanez a mozdulat a lista görgetését is jelentheti. A húzás ezért csak akkor kezdődik, ha az ujj 150 ezredmásodpercig egy helyben maradt, különben a mozdulat görgetésnek számít.

5. fejezet

Tesztelés és használat

A fejezet első fele a szerkesztő ellenőrzésének módját mutatja be, a második fele az elkészült alkalmazás használatát.

5.1. Tesztelési stratégia

A tesztelés a 3.4. szakaszban leírt rétegeket követi. Minél lejjebb van egy kódrészlet a rétegek között, annál egyszerűbb és gyorsabb tesztelni, ezért a tesztek többsége az alsó rétegekre irányul. A geometriai számításokat, az útvonalak feldolgozását, a színértékek értelmezését, az SVG-importálást és -exportálást a tesztek közvetlenül, bemenetek és elvárt kimenetek alapján ellenőrzik.

Az eszközök tesztjei mesterséges `PointerInfo` eseményeket küldenek, majd a dokumentum és az előzmények állapotát vizsgálják. Egy téglalap rajzolásának tesztje például egy lenyomást, néhány mozgatást és egy felengedést ad át a téglalap eszköznek, és ellenőrzi, hogy pontosan egy új alakzat és egy új előzménybejegyzés keletkezett. A felület részeit, a vásznat, a paneleket, a színválasztót és a párbeszédablakokat, végül a teljes alkalmazást a tesztek a jsdom környezetében jelenítik meg [31], és DOM-eseményekkel kezelik.

A jsdom nem végez valódi elrendezést és megjelenítést, így például az elemek mérete és a szöveg szélessége nem mérhető vele. Ezeket a tesztek egyszerű közelítéssel helyettesítik, a valódi megjelenést pedig böngészőben ellenőriztem. Az automatikus teszteket valódi böngészőben végzett kézi tesztelés egészítette ki, és az ennek során talált hibákhoz a javítás előtt egy-egy automatikus teszt készült.

5.2. Automatikus tesztek

A tesztek a Vitest keretrendszerrel futnak [70], a `tests` mappában, témák szerint öt almappába rendezve. A tesztek témakörök szerinti megoszlása az 5.1. táblázatban látható.

A forráskódban 409 teszteset szerepel. Futtatáskor a keretrendszer 447 tesztet jelez, mert a véletlenszerű tesztek egyik tesztesete negyven különböző kezdőértékkel fut le. A teljes tesztkészlet egy átlagos fejlesztői gépen 30-40 másodperc alatt fut le, és minden módosítás után könnyen lefuttatható. Az 5.2. táblázat azt mutatja, mely tesztfájlok ellenőrzik a 3.1. táblázat követelményeit.

5.1. táblázat. Az automatikus tesztek elosztása

Mappa	**Témakör**	**Fájl**	**Teszteset**
`model`	alakzatok, útvonalak, csoportok, színátmenetek, szaggatás, előzmények, projektfájl, beállítások, véletlenszerű tesztek	12	114
`tools`	rajzolás, kijelölés, csomópontok, illesztés, logikai műveletek, olló, foltecset	7	71
`io`	SVG-importálás és -exportálás, raszteres export, képek, vágólap	6	64
`fonts`	Google Fonts kérések, helyi betűtípusok, beágyazás, szöveg útvonallá alakítása, betűtípus-választó	7	79
`ui`	vászon, panelek, színválasztó, párbeszédablakok, nézet, teljes alkalmazás, regressziós tesztek	8	81
Összesen		**40**	**409**

5.2. táblázat. A funkcionális követelmények és az őket ellenőrző tesztfájlok

Az.	**Terület**	**Tesztfájlok**
F1	Elrendezés	`ui/app`, `ui/panels`, böngészős ellenőrzés
F2	Rajzolás	`tools/drawing`, `ui/canvas`
F3	Tulajdonságok	`model/shapes`, `tools/select`, `ui/panels`
F5	Szöveg	`fonts/text`, `fonts/outline`, `fonts/textOutline`, `ui/canvas`
F6	Színek	`model/gradient`, `model/dash`, `ui/color-picker`
F7	Útvonalak	`model/path`, `model/strokeToPath`, `tools/edit`, `tools/boolean`, `tools/scissors`
F8	Illesztés, igazítás	`tools/snap`
F9	Objektumok	`model/groups`, `model/lock`
F10	Kijelölés	`tools/select`, `ui/canvas-tools`
F11	Vágólap	`io/clipboard`
F12	Nagyítás	`ui/view`, böngészős ellenőrzés
F13	Visszavonás	`model/history`, `model/fuzz`
F14	Mentés	`model/project`, `model/preferences`
F15	Import, export	`io/import`, `io/serializer`, `io/export`, `io/images`, `fonts/embed`

Néhány jellemző eset a tesztek közül:

- egy elforgatott téglalap importálás után elforgatott téglalap marad, egy nyírt téglalap viszont útvonallá alakul;
- egy csoport exportálás és újbóli importálás után is megtartja a forgatását és az átlátszóságát;
- két kör egyesítése néhány görbe csomópontot ad, nem sokszöget;
- a szöveg útvonallá alakításakor az „o” betű két kontúrt kap, egy külsőt és egy belső lyukat, a magyar „ő” és „ű” betű pedig helyesen jelenik meg;
- egy ismeretlen betűtípusú szöveget a szerkesztő nem alakít útvonallá, a szöveg szöveg marad.

5.3. Véletlenszerű tesztelés

A kézzel írt tesztek azokat az eseteket ellenőrzik, amelyekre a fejlesztő gondolt. A szerkesztőben viszont a műveletek tetszőleges sorrendben követhetik egymást, és egyes hibák csak ritka kombinációkban jelentkeznek, például egy csoportba belépve, egy forgatás és egy visszavonás után. Ezek felderítésére véletlenszerű műveletsorozatokat végrehajtó teszt készült, angol nevén *fuzzing*. A módszer Miller és társai 1990-es vizsgálatából ered, amelyben véletlenszerű bemenettel a UNIX-segédprogramok negyedét-harmadát sikerült összeomlasztani [57].

A teszt egy kezdőértékkel inicializált álvéletlenszám-generátor alapján súlyozottan választ a műveletek közül. Alakzatokat rajzol az eszközökkel, mozgat, méretez, forgat, csoportosít, csoportba lép be, csomópontokat szerkeszt, stílust módosít, visszavon és megismétel. Az ellenőrzés a QuickCheck eszközzel ismertté vált tulajdonságalapú tesztelést követi [6], vagyis nem egy konkrét kimenetet vár, hanem olyan állításokat, amelyeknek minden állapotban igaznak kell lenniük. Minden lépés után néhány olcsón ellenőrizhető tulajdonságot vizsgál, például hogy a kijelölés pontosan a belépett csoport szintjén lévő alakzatokra vonatkozik. Néhány lépésenként és a futás végén költségesebb ellenőrzések is lefutnak:

- a projektfájlba mentett, majd visszatöltött dokumentum ugyanazokat az alakzatokat adja ugyanott
- az exportált SVG minden útvonal csomópontjait tartalmazza
- az összes lépés visszavonása után a kiinduló dokumentum áll vissza

Alapértelmezés szerint negyven kezdőérték fut le, egyenként százötven lépéssel, és egy további futás hétszáz lépéssel, amely túllépi az előzmények kétszáz lépéses korlátját. Hiba esetén a teszt kiírja a kezdőértéket és az utolsó műveleteket, a hibás sorozat tehát pontosan megismételhető.

5.4. Betűtípusok tesztelése hálózat nélkül

A betűtípusokkal kapcsolatos kód nagy része a Google Fonts és a Fontsource szolgáltatásainak küld kéréseket. A tesztek nem használhatják a valódi hálózatot, mert akkor lassúak és megbízhatatlanok lennének. Egy segédmodul ezért helyettesíti a szolgáltatásokat, és ott, ahol ez számít, a valódi API viselkedését utánozza. Egy nem létező stílus kérését az egész kérés elutasításával válaszolja meg, a `text` paraméteres kérésre egyetlen `@font-face` blokkot ad, a paraméter nélküli kérésre pedig írásrendszerenként egyet [24]. Betűtípusfájlként a tesztkészletben tárolt kis betűtípusfájlokat küldi vissza.

5.5. Kézi tesztelés és regressziós tesztek

A kézi tesztelés során a felület minden részét végigpróbáltam, és a talált hibákat egy hibalistába gyűjtöttem. A hibák egy része csak a komponensek együttműködésében jelentkezett, ezért a hozzájuk tartozó tesztek a teljes vásznat jelenítik meg. Az 5.3. táblázat a hibalista néhány jellemző bejegyzését és a hozzájuk tartozó teszteket mutatja. A tesztek neve a hiba azonosítójával kezdődik, egy hiba újbóli megjelenése tehát azonnal azonosítható.

5.3. táblázat. Hibák a kézi tesztelésből és a hozzájuk tartozó regressziós tesztek

Azonosító	**A teszt által ellenőrzött viselkedés**
B-01	Az exportált SVG jól formált XML, és nem függ külső betűtípustól.
B-02	Egy csomópont húzása közben a csomópontok és a fogantyúk a görbével együtt mozognak.
B-03	A toll eszközzel az első pontra lenyomva és húzva kattintásként viselkedik, a későbbi pontok viszont sima csomóponttá húzhatók.
B-04	Az útvonal bezárásának előnézete pontosan azt az alakzatot mutatja, amelyet a kattintás létrehoz.
B-05	Rajzolás és kijelölés után a mozgatás, a méretezés és a forgatás a kijelölő keretet is mozgatja.
B-06	A kijelölés megszüntetése, legyen az kattintás az üres vászonra, Esc vagy dupla kattintás, eltünteti a kijelölő keretet.

5.6. Böngészős ellenőrzés és a kész alkalmazás mérete

A fejlesztés során az alkalmazást folyamatosan egy Chromium-alapú asztali böngészőben használtam, a kézi tesztelés is ott történt. A jsdom által nem vizsgálható viselkedés, például a valódi elrendezés, a szöveg mérése, a raszteres export és az érintéses mozdulatok ellenőrzésének egy részét a Playwright eszközzel szkriptekbe foglaltam [55]. A Playwright programból vezérel egy Chromium böngészőt, asztali és telefonos képernyőméretre is beállítható, és érintéses eseményeket is küldhet. A dolgozat képernyőképei is így készültek. A felületet Firefox böngészőben is kipróbáltam, ott is helyesen működik, Safari böngészőben azonban nem teszteltem. A helyi betűtípusok elérése a Chromium-alapú böngészőkre korlátozódik, ezt a 4.10. szakasz tárgyalja.

Az éles változatot a Vite csomagolja. A keletkező fő JavaScript-fájlokat az 5.4. táblázat sorolja fel. A nagy könyvtárak dinamikus importtal külön fájlba kerülnek, és

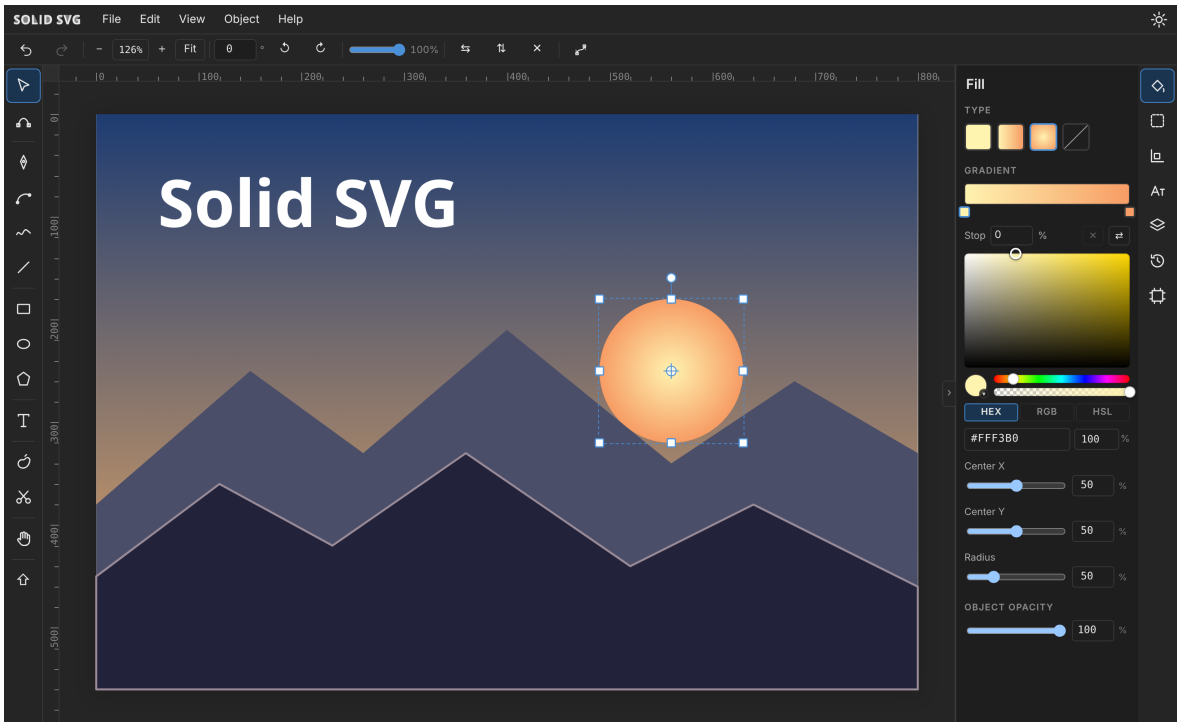
csak az első használatukkor töltődnek be: a paper.js az első logikai műveletnél vagy vágásnál, a fontkit és a WOFF2 kibontó az első útvonallá alakításnál. Az alkalmazás indulásakor ezért csak a fő fájlt kell letölteni, amely tömörítve körülbelül 160 kB.

5.4. táblázat. Az éles változat fő JavaScript-fájljai

Fájl	Tartalom	Méret	Tömörítve
`index`	a szerkesztő kódja	504,2 kB	160,8 kB
`paper-core`	paper.js	318,9 kB	112,3 kB
`browser-module`	fontkit	363,0 kB	150,5 kB
`decompress`	WOFF2 kibontó	295,2 kB	127,0 kB

5.7. Az alkalmazás használata

Indításkor az alkalmazás a legutóbbi automatikus mentést tölti vissza, ennek hiányában üres lappal nyílik meg. Az 5.1. ábra a szerkesztőt asztali képernyőn mutatja. A rajzon egy kijelölt kör látható a nyolc méretező fogantyúval, a felette lévő forgató fogantyúval és a közepén a forgatási középponttal. A jobb oldali kitöltés panelen a kör sugárirányú színátmenete szerkeszthető. A színátmenet sávja alatt a színpontok, alattuk a színválasztó, legalul pedig az átmenet középpontja és sugara látható.



5.1. ábra. A szerkesztő felülete asztali képernyőn, egy kijelölt alakzattal és a kitöltés panellel

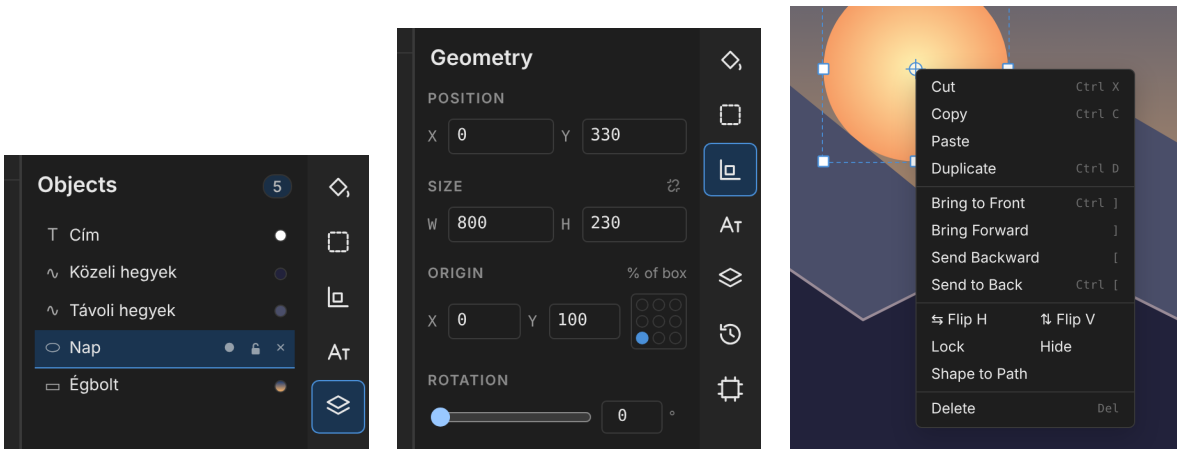
Rajzolás. Az eszköz a bal oldali eszköztárról vagy a billentyűjével választható ki. A téglalap, az ellipszis és a vonal húzással rajzolható, Shift billentyűvel négyzet, kör,

illetve 45 fokos lépésekben elforgatott vonal készül. A sokszög eszköz kattintásonként egy csúcsot ad hozzá, a toll és a körív a 4.6. szakaszban leírtak szerint működik. A szöveg eszközzel a vászonra kattintva közvetlenül lehet gépelni. Az eszközök billentyűit az 5.5. táblázat foglalja össze.

5.5. táblázat. Az eszközök és billentyűik

Eszköz	**Bill.**	**Eszköz**	**Bill.**	**Eszköz**	**Bill.**
Kijelölés	V	Szabadkézi rajz	F	Szöveg	T
Csomóponteszköz	E	Vonal	L	Foltecset	Shift+B
Toll	B	Téglalap	R	Olló	C
Körív	A	Ellipszis	O	Nézet mozgatása	H
		Sokszög	P		

Objektumok és geometria. Az objektumok panel, a felületen *Objects*, a dokumentum fáját mutatja, ez látható az 5.2. ábra bal oldalán. Minden sorban ott van az alakzat típusának ikonja, a neve és a kitöltése, a kijelölt sorban pedig a zárolás és az elrejtés gombja is. A sorok húzással rendezhetők, és egy sort egy csoportra húzva az alakzat a csoportba kerül. Az ábra közepén a geometria panel, a felületen *Geometry*, a pozíciót, a méretet, a forgatási középpontot és a forgatást mutatja. A forgatási középpont a határoló téglalap százalékában vagy a mellette lévő kilenc pont egyikével adható meg. Az ábrán a bal alsó sarokba került, a kijelölt hegyek tehát a bal alsó sarkuk körül fordulnának el.



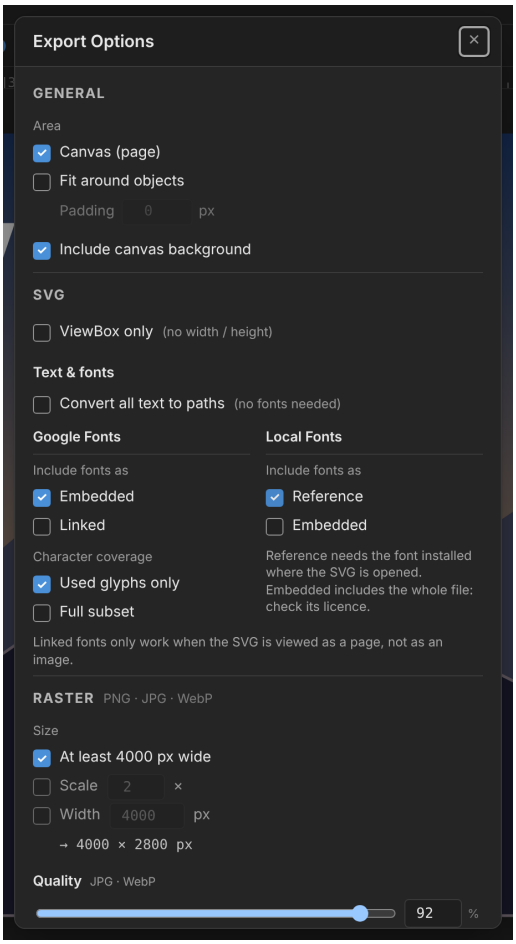
Objektumok panel Geometria panel Helyi menü

5.2. ábra. Az objektumok panel, a geometria panel és egy alakzat helyi menüje

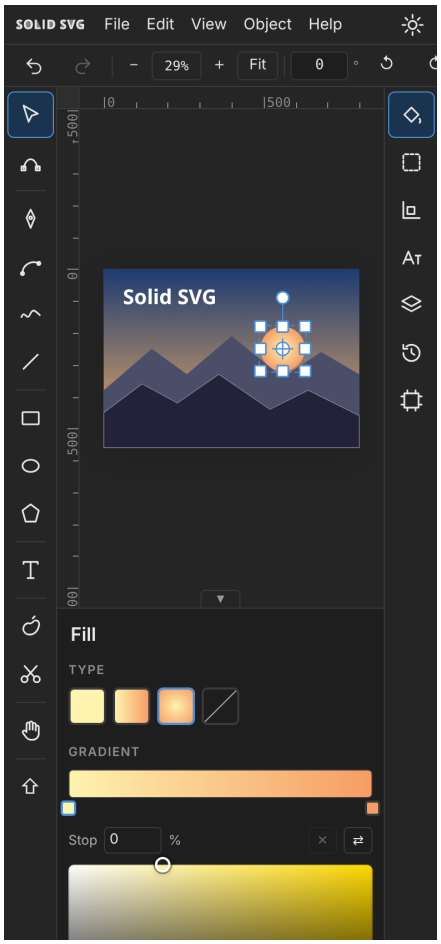
Helyi menü. Egy alakzatra jobb egérgombbal kattintva vagy érintőképernyőn hosszan megérintve megjelenik a helyi menü, az 5.2. ábra jobb oldalán. A menüben csak az adott helyzetben elérhető műveletek szerepelnek: a vágólap műveletei, a sorrend módosítása, a tükrözés, a zárolás, az elrejtés, az útvonallá alakítás és a törlés. A billentyűkombinációk a menüpontok mellett is láthatók.

Mentés és exportálás. A *File* menü kínálja az új dokumentumot, a projekt vagy SVG-fájl megnyitását, a projekt mentését, SVG-fájlok és képek beillesztését a meglévő rajzba, valamint az exportálást SVG és raszteres formátumba. Az exportálás beállításai külön párbeszédablakban adhatók meg, amelyet az 5.3. ábra mutat. Itt választható ki, hogy a lap vagy csak a rajzolt alakzatok környezete kerüljön a fájlba, hogy a háttér is része legyen-e, és hogy a szöveg a 4.11. szakaszban leírt módok közül melyikkel hagyja el a szerkesztőt. A raszteres képek mérete és minősége is itt állítható.

Telefonon. Keskeny kijelzőn a felület az 5.4. ábrán látható módon rendeződik át. A tulajdonságpanel a vászon alatt jelenik meg, az eszköztár és a tulajdonságsáv a két oldalon marad. Az eszköztár alján lévő nyíl gomb a Shift billentyűt helyettesíti, bekapcsolva például a téglalap eszköz négyzetet rajzol, és a kijelölés bővíthető. A nézet két ujjal nagyítható és mozgatható.



5.3. ábra. Az exportálási beállítások párbeszédablakja



5.4. ábra. A szerkesztő telefonon, nyitott tulajdonságpanellel

6. fejezet

Az eredmények értékelése

Ez a fejezet azt vizsgálja, mennyiben teljesültek a 2.2.7. alszakasz célkitűzései és a 3.1. szakasz követelményei, mit mutatnak a két legfontosabb funkció mérései, hogyan viszonyul az elkészült szerkesztő a 2.2. szakaszban vizsgált alkalmazásokhoz, és milyen nem várt tapasztalatokkal járt a fejlesztés.

6.1. A követelmények teljesülése

A 6.1. táblázat a funkcionális követelmények teljesülését foglalja össze. A követelmények többsége teljesült, kettő részben, a korlátozásokat a megjegyzés oszlop nevezi meg.

6.1. táblázat. A funkcionális követelmények teljesülése

Az.	**Terület**	**Teljesülés**	**Megjegyzés**
F1	Elrendezés	teljesült	
F2	Rajzolás	teljesült	
F3	Tulajdonságok	teljesült	több elforgatott alakzat nem arányos méretezése közelítés
F4	SVG-kód	teljesült	a kódszerkesztő a terv szerint kimaradt
F5	Szöveg	részben	a vegyes írásirányú szöveg nem alakítható útvonallá
F6	Színek	teljesült	
F7	Útvonalak	teljesült	a körvonal útvonallá alakítása sokszöget ad
F8	Illesztés, igazítás	teljesült	
F9	Objektumok	teljesült	
F10	Kijelölés	teljesült	
F11	Vágólap	teljesült	
F12	Nagyítás	teljesült	
F13	Visszavonás	teljesült	legfeljebb kétszáz lépés
F14	Mentés	teljesült	a böngésző tárolójának mérete korlátozott
F15	Import, export	részben	minták, vágóútvonalak, maszkok, szűrők, jelölők, keverési módok és stíluslapok nélkül, figyelmeztetéssel

A hat célkitűzés közül az első, a második, az ötödik és a hatodik teljesült. A szerkesztő telepítés, regisztráció és szerver nélkül fut a böngészőben, a rajzoló és szerkesztő eszközei között ott van a teljes útvonalszerkesztés és mind az öt logikai művelet, a kimenete szabványos SVG vagy raszteres kép, és telefonon is kezelhető. A harmadik cél

a vegyes írásirányú szövegek kivételével teljesült. A negyedik cél, az importált fájlok korlátainak jelzése a 6.2.1. alszakasz mérése szerint a vizsgált fájlok döntő többségénél teljesült, maga az importálás azonban nem kezel minden SVG-szerkezetet. A 4.2. táblázat felsorolja, melyek ezek, így egy bemeneti fájlról előre eldönthető, milyen korlátozásokkal szerkeszthető.

A nem funkcionális követelmények is teljesültek. A geometriai számításokat, az előzményeket és a fájlkezelést 447 automatikus teszt ellenőrzi, az érintéses fogantyúk mérete megfelel a WCAG 2.2 célméretre vonatkozó előírásának, a sérült projektfájlok pedig nem okoznak hibát. A felület Chromium-alapú böngészőkben és Firefoxban működik, Safari böngészőben nem teszteltem.

6.2. Mérések

Két célkitűzés teljesülése számszerűen is ellenőrizhető. Az importálónak minden olyan szerkezetről szólnia kell, amelyet nem tud pontosan átvenni, a szöveg útvonallá alakításának pedig a képernyőn látott szöveget kell visszaadnia. Mindkét mérés a fejlesztői szerveren futó szerkesztő saját moduljait hívja egy Playwright programkönyvtárral vezérelt Chromium böngészőben, hálózati hozzáférés nélkül, így pontosan a dolgozatban leírt kódot méri. A mérőszkriptek és a felhasznált fájlok listája a mellékelt adathordozón, a `program/eval` mappában található.

6.2.1. Az importálás pontossága valós fájlokon

Az importáló tesztjei kézzel írt, rövid SVG-részleteket használnak. Hogy valódi, más programokkal készült fájlokon hogyan viselkedik, azt hét forrásból vett 140 fájlon vizsgáltam. Az Inkscape 1.2.2 példafájljai és angol nyelvű oktatóanyagai szándékosan sokféle SVG-szerkezetet mutatnak be, köztük szűrőket, maszkokat és jelölőket. Mellettük négy ikonkészletből és egy zászlógyűjteményből húsz-húsz véletlenszerűen kiválasztott fájl került a mintába, ezek a webes felhasználás jellemző, egyszerűbb fájljai. A csomagok pontos változatát a 6.2. táblázat adja meg.

Minden fájlt a szerkesztő importált, majd az eredményt szabványos SVG-ként exportálta. Az eredeti és az exportált fájlt a böngésző fehér háttérre, 400 képpontos hosszabbik oldallal képpé rajzolta. Egy képpontot akkor tekintettem a rajzhoz tartozónak, ha valamelyik színcsatornája legalább 40 egységgel sötétebb a fehérnél valamelyik képen, és akkor eltérőnek, ha a két kép színcsatornáinak legnagyobb különbsége 64-nél nagyobb. A mérőszám az eltérő képpontok aránya a rajzhoz tartozó képpontokhoz képest. Az 1% alatti eltérés jellemzően a vonalak élsimításából adódik, szemmel nem látható. A 6.2. táblázat az eredményeket foglalja össze.

A négy ikonkészlet mind a nyolcvan fájlja figyelmeztetés nélkül és gyakorlatilag változatlanul kerül át. A zászlók közül tizennyolc 1% alatti eltéréssel jelenik meg, tizenegynél figyelmeztetés szól, öt esetben vágóútvonal, nyolc esetben körvonalas útvonal szétválasztása miatt. Az Inkscape fájljainál a nagy eltérés várható, hiszen ezek éppen azokat a szerkezeteket mutatják be, amelyeket a szerkesztő nem támogat. A teljes mintában a leggyakoribb figyelmeztetés a szűrőkről szól, tizenkilenc fájlban, ezt a körvonalas útvonalak szétválasztása követi tizenöt fájlban.

A célkitűzés szempontjából az a döntő, hogy figyelmeztetés nélkül nagy eltérés ne fordulhasson elő. A 139 fájlból három tér el 5%-nál jobban úgy, hogy az importáló nem jelez semmit. Kettőnél csak a vékony vonalak és az apró szöveg élsimítása különbözik,

6.2. táblázat. Az importálás pontossága forrásonként; az eltérés az eltérő képpontok aránya a rajz képpontjaihoz képest

Forrás	**Fájl**	**Figyelmeztetés nélkül**	**1% alatti eltérés**	**Medián eltérés**	**Legnagyobb eltérés**
Inkscape 1.2.2, példák	28	4	5	21,5%	81,9%
Inkscape 1.2.2, oktatóanyagok	11	0	0	2,8%	53,6%
Bootstrap Icons 1.13.2	20	20	20	0,0%	0,1%
Tabler Icons 3.51.0	20	20	20	0,0%	0,6%
Simple Icons 16.37.0	20	20	20	0,0%	0,2%
Twemoji 15.0.0	20	20	20	0,0%	0,0%
flag-icons 7.6.0	20	9	18	0,0%	3,4%
Összesen	**139**	**93**	**103**		

a két kép szemmel nem különböztethető meg. A harmadiknál, az Inkscape `stars.svgz` példájánál egyes összetett útvonalak kitöltése valóban eltér, ennek okát nem sikerült feltárnom. Ez az importáló egyetlen ismert, jelzés nélküli hibája a mintában.

6.2.2. A szöveg útvonallá alakításának pontossága

A szöveg útvonallá alakításának mérésekor egy egysoros, 72 képpontos szöveg került a vászonra 100%-os nagyításon. A szerkesztő vásznáról a böngésző által kirajzolt szövegről, majd az útvonallá alakítás után az eredményről is képernyőkép készült. A két képet az importálás méréséhez hasonlóan vetettem össze, rajzhoz tartozónak a 215-nél sötétebb szürke árnyalatot tekintve. A szöveg, az *Árvíztűrő tükörfúrógép AVATAR fi*, magyar ékezetes betűket, alávágott betűpárokat és egy *fi* ligatúrát is tartalmaz. A mérés az öt általános betűcsaládra terjed ki, amelyekhez a szerkesztő saját betűtípust szállít. A Google Fonts és a helyi betűtípusok pontosságát ez a mérés nem vizsgálja. Az eredményeket a 6.3. táblázat tartalmazza.

6.3. táblázat. A szöveg útvonallá alakításának eltérése a képernyőn látott szövegtől

Betűcsalád	**Szállított betűtípus**	**Normál (400)**	**Félkövér (700)**
`serif`	Noto Serif	0,14%	1,77%
`sans-serif`	Noto Sans	0,09%	0,94%
`monospace`	Noto Sans Mono	0,09%	0,17%
`fantasy`	Anton	0,03%	0,02%
`cursive`	Comic Neue	0,22%	0,07%

Az eltérés minden esetben 2% alatt marad, a legnagyobb a félkövér Noto Serif betűtípusnál. A mintaszöveg a hiányzó karakterek pótlását is próbára teszi. A Comic Neue betűtípusból hiányzik az *ő* és az *ű*, ezeket a szerkesztő a képernyőn és az útvonallá alakításkor is a Noto Sans betűtípusból veszi. A böngészőhöz hasonlóan az útvonallá alakítás is karakterenként választ betűtípust, így a pótolt betű utáni szóköz a Comic Neue betűtípusé marad, és a sor további része nem csúszik el.

6.3. Összevetés a vizsgált szerkesztőkkel

A 6.4. táblázat a 2.2. táblázat szempontjai szerint mutatja be az elkészült szerkesztőt, és mindegyik szempontnál megadja, melyik vizsgált alkalmazáshoz áll a legközelebb.

6.4. táblázat. Az elkészült szerkesztő az összehasonlítás szempontjai szerint

Szempont	**Solid SVG**	**Összevetés**
Platform	webes, telepítés és regisztráció nélkül	az SVG-Edithez hasonlóan ingyenes és böngészőben fut
Elrendezés	klasszikus, helyzetfüggő parancssávval, telefonon alulról nyíló panellel	a klasszikus elrendezésű Inkscape, SVG-Edit és Boxy SVG csoportjába tartozik
SVG-kód	nincs kódnézet	elmarad az Inkscape, az SVG-Edit és a Boxy SVG mögött
Szöveg	többsoros, Google Fonts és helyi betűtípusok, útvonallá alakítás	az SVG-Editnél többet tud, útvonalra illesztés nincs, ebben az Inkscape és a Boxy SVG előtte jár
Színek	HEX, RGB, HSL, átlátszóság	a Figmához és az SVG-Edithez hasonló, CMYK nincs
Színátmenet, minta	lineáris és sugárirányú színátmenet	az SVG-Edit szintje, minta nincs
Útvonalak	teljes csomópont-szerkesztés, körív, olló	az Inkscape és a Boxy SVG alapfunkcióival egyező
Logikai műveletek	unió, különbség, metszet, kizárás, felosztás	megegyezik az Inkscape és a Boxy SVG kínálatával
Illesztés	rács, segédvonalak, más alakzatok	az Inkscape-hez és a Boxy SVG-hez hasonló
Objektumok	objektumfa, zárolás, elrejtés, külön rétegek nélkül	a Figma rétegfájához hasonló
Előzmények, mentés	előzménypanel, automatikus mentés a böngészőbe, projektfájl	az Inkscape-hez hasonló, felhőmentés nincs

Az összevetés szerint a szerkesztő az útvonalak szerkesztésében és a logikai műveletekben lefedi az Inkscape és a Boxy SVG alapfunkcióit, és ezen a téren többet nyújt a vizsgált egyszerűbb webes eszközöknél, az SVG-Editnél és a Mediamodifiernél. Az Inkscape ugyanakkor számos további eszközt kínál, például útvonaleffektusokat és hálós színátmenetet, ezek nem voltak célja a szerkesztőnek. A kódnézet, a minták, a szöveg útvonalra illesztése és a CMYK színmegadás hiányzik belőle, ezekben a két nagy tudású szerkesztő előtte jár. Saját megoldásként emelném ki a szöveg útvonallá alakítását, amely a böngészővel azonos szövegszedést használ, és a hiányzó betűtípust sosem helyettesíti csendben egy másikkal, valamint az importálót, amely minden kihagyott vagy közelített szerkezetet megnevez.

6.4. Nem várt tapasztalatok

A fejlesztés során több olyan probléma is felmerült, amelyet a tervezéskor nem láttam előre. A legtöbb munkát a szöveg útvonallá alakítása adta, mert a betűk elrendezésének pontosan a böngészőét kell követnie, és ehhez az először kipróbált könyvtár nem bizonyult elég pontosnak (4.10. szakasz). A külső szolgáltatások és a böngészők is tartogattak meglepetést. A Google Fonts API egy nem létező stílus miatt az egész kérést elutasítja, az iOS Safari nem küldi el a helyi menü eseményét, a két ujjas nagyítás első érintése pedig egy felesleges csomópontot hozna létre, ha azonnal eljutna a toll eszközhöz (4.13. szakasz).

Tanulságos volt az is, hogy a 6.2. szakasz mérései olyan hibákra is fényt derítettek, amelyeket a több száz automatikus teszt nem észlelt, hiszen azok csak azokat a bemeneteket ellenőrzik, amelyekre fejlesztőként én magam is gondoltam. A valós fájlok mutatták meg, hogy néhány nem támogatott szerkezetre, például a szűrőkre és a maszkokra, külön figyelmeztetés kell, és hogy a körvonal nélküli ikonok vegyes részútvonalait nem szabad szétválasztani, mert az eltünteti a lyukakat. A szövegmérésből derült ki, hogy a pótolt karakter utáni szóköz betűtípusát is a böngésző szabálya szerint kell megválasztani. Ezekhez az esetekhez automatikus tesztek is készültek.

6.5. Szakmai jelentőség és alkalmazhatóság

Az elkészült szerkesztő a gyakorlatban is használható ikonok, logók, diagramok és egyszerű illusztrációk készítésére és módosítására, telepítés nélkül, asztali gépen és telefonon egyaránt. A kimenete szabványos SVG, ezért az eredmény más programokban is tovább szerkeszthető. Az alkalmazás egyes részei önállóan is hasznosak lehetnek. Az importáló, amely minden kihagyott szerkezetről figyelmeztet, egy SVG-fájl szerkeszthetőségének előzetes vizsgálatára is alkalmas, a szöveg útvonallá alakítása pedig olyan esetekben is használható, amikor egy SVG-fájlnak betűtípusok nélkül, minden környezetben ugyanúgy kell megjelennie. A hiányzó funkciók, elsősorban a kódnézet és a minták, maszkok és szűrők támogatása, a 7. fejezetben felsorolt továbbfejlesztési lehetőségek közé tartoznak.

7. fejezet

Összefoglalás

Szakdolgozatomban a Solid SVG webalapú SVG-szerkesztőt terveztem és valósítottam meg. Az SVG grafikus elemeinek áttekintése és öt elterjedt szerkesztő, az Inkscape, az SVG-Edit, a Boxy SVG, a Figma és a Mediamodifier összehasonlítása alapján hat célt tűztem ki: a szerkesztő telepítés és regisztráció nélkül fusson a böngészőben, kínáljon teljes értékű útvonalszerkesztést logikai műveletekkel, kezelje a többsoros szöveget, jelezze az importált fájlok korlátait, exportáljon szabványos SVG-t és raszteres képet, és legyen telefonon is kezelhető. A célokat tizenöt funkcionális követelményre bontottam, az alkalmazást Svelte keretrendszerrel, TypeScript nyelven készítettem el.

Az elkészült szerkesztő szerver nélkül fut, tizenhárom eszközzel rajzol és szerkeszt, a logikai műveletek eredménye görbékből álló, tovább szerkeszthető alakzat, a szöveg pedig a képernyőn látottnak megfelelően alakítható útvonallá. A tervezés legfontosabb döntései a böngészőre bízott találatvizsgálat, a pillanatképekre épülő előzménykezelés és az egyirányú adatfolyam voltak, ezek tették lehetővé, hogy a működést 447 automatikus teszt ellenőrizze.

A követelmények közül tizenhárom teljesült és kettő részben. Két mérés a legfontosabb célkitűzéseket számszerűen is igazolja. Az útvonallá alakított szöveg képpontjainak legfeljebb 2%-a tér el a képernyőn látottól, 139 valós SVG-fájl importálásakor pedig a négy ikonkészlet minden fájlja változatlanul került át, és a nagyobb eltérésekről három fájl kivételével figyelmeztetés szólt. A legfontosabb hiányosságok a kódnézet, a minták, vágóútvonalak, maszkok és szűrők kezelése, a szöveg útvonalra illesztése és a vegyes írásirányú szöveg útvonallá alakítása. Ezek, valamint egy magyar nyelvű felület, a továbbfejlesztés irányai.

A munka során a vektorgrafika geometriájától a betűtípusok működéséig sok olyan területet ismertem meg mélyebben, amelyekkel korábban csak felhasználóként találkoztam. A legtöbbet abból tanultam, amikor egy egyszerűnek tűnő funkcionalitás, például a szöveg útvonallá alakítása, a megvalósítás közben bonyolultnak bizonyult.

Summary

This thesis presents the design and implementation of Solid SVG, a web-based editor for Scalable Vector Graphics. The work started with an overview of the graphical elements of SVG and a comparison of five widely used editors: Inkscape, SVG-Edit, Boxy SVG, Figma and Mediamodifier. Based on this comparison, six goals were set. The editor should run in the browser without installation, registration or a server; it should offer full path editing with boolean operations that keep curves as curves; it should handle multi-line text and convert it to paths exactly as it appears on screen; it should report which structures of an imported SVG file it cannot handle; it should export standard SVG and raster images; and it should be usable on phones and tablets. These goals were broken down into fifteen functional requirements.

The editor was built with the Svelte framework and TypeScript. Its architecture separates pure geometric functions, the application state and the editing actions, with a one-way data flow from the tools and components to the document. Hit-testing is delegated to the browser, the undo history is based on immutable snapshots, and every change is a single named history step. The thesis describes the more involved parts of the implementation: rotation-aware resizing, three-point circular arcs approximated by cubic Bézier curves, freehand drawing simplified with the Ramer–Douglas–Peucker algorithm, boolean operations and knife cuts with paper.js, stroke-scaled dash patterns, text-to-path conversion with full OpenType shaping, SVG import with explicit warnings, and touch input including pinch zoom.

The behaviour is verified by 447 automated tests, including randomised action sequences, and by manual testing in Chromium-based browsers and Firefox. Two measurements complete the evaluation. Importing 139 real SVG files from seven sources and comparing the rendered original with the rendered export showed that all files of four icon sets survive practically unchanged, and that every larger difference but three is reported to the user; converting text to paths changes at most 2% of the text's pixels. Thirteen requirements were met in full and two in part. The main missing features are a code view, support for patterns, clipping paths, masks and filters, text on a path, and bidirectional text in path conversion; these, together with a Hungarian user interface, are the directions for further development.

Through this work, I gained a deeper understanding of many areas, from the geometry of vector graphics to how fonts work, that I had previously encountered only as a user. I learned the most when a seemingly simple functionality, such as converting text into a path, turned out to be complicated to implement.

Források

[1] Thomas Berlage. „A selective undo mechanism for graphical user interfaces based on command objects". *ACM Transactions on Computer-Human Interaction* 1.3 (1994), 269–294. old. DOI: 10.1145/196699.196721.

[2] Boxy SVG. *Boxy SVG*. URL: https://boxy-svg.com/ (elérés dátuma 2026. 11. 11.).

[3] Britannica. *Freemium*. URL: https://www.britannica.com/topic/freemium (elérés dátuma 2026. 11. 08.).

[4] Britannica. *Raster graphics*. URL: https://www.britannica.com/technology/raster-graphics (elérés dátuma 2026. 11. 08.).

[5] Edwin Catmull és Raphael Rom. „A class of local interpolating splines". *Computer Aided Geometric Design*. Szerk. Robert E. Barnhill és Richard F. Riesenfeld. Academic Press, 1974, 317–326. old. DOI: 10.1016/B978-0-12-079050-0.50020-5.

[6] Koen Claessen és John Hughes. „QuickCheck: A lightweight tool for random testing of Haskell programs". *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. ACM, 2000, 268–279. old. DOI: 10.1145/351240.351266.

[7] David H. Douglas és Thomas K. Peucker. „Algorithms for the reduction of the number of points required to represent a digitized line or its caricature". *The Canadian Cartographer* 10.2 (1973), 112–122. old.

[8] James R. Driscoll és tsai. „Making data structures persistent". *Journal of Computer and System Sciences* 38.1 (1989), 86–124. old. DOI: 10.1016/0022-0000(89)90034-2.

[9] Gershon Elber, In-Kwon Lee és Myung-Soo Kim. „Comparing offset curve approximation methods". *IEEE Computer Graphics and Applications* 17.3 (1997), 62–71. old. DOI: 10.1109/38.586019.

[10] Gerald Farin. *Curves and Surfaces for CAGD: A Practical Guide*. 5. kiad. San Francisco: Morgan Kaufmann, 2002.

[11] Figma. *Access design tools from the toolbar*. URL: https://help.figma.com/hc/en-us/articles/360041064174 (elérés dátuma 2026. 11. 11.).

[12] Figma. *Add a font to Figma Design*. URL: https://help.figma.com/hc/en-us/articles/360039956894 (elérés dátuma 2026. 11. 11.).

[13] Figma. *Boolean operations*. URL: https://help.figma.com/hc/en-us/articles/360039957534 (elérés dátuma 2026. 11. 11.).

[14] Figma. *Color models in Figma Design*. URL: https://help.figma.com/hc/en-us/articles/360043042113 (elérés dátuma 2026. 11. 11.).

[15] Figma. *Figma*. URL: https://www.figma.com/ (elérés dátuma 2026. 11. 08.).

[16] Figma. *Navigating UI3: Figma's new UI*. URL: https://help.figma.com/hc/en-us/articles/23954856027159 (elérés dátuma 2026. 11. 11.).

[17] Figma. *Vector networks*. URL: https://help.figma.com/hc/en-us/articles/360040450213 (elérés dátuma 2026. 11. 11.).

[18] Figma. *View a file's version history*. URL: https://help.figma.com/hc/en-us/articles/360038006754 (elérés dátuma 2026. 11. 11.).

[19] Mike Fogel. *polygon-clipping*. URL: https://www.npmjs.com/package/polygon-clipping (elérés dátuma 2026. 11. 12.).

[20] foliojs. *fontkit: An advanced font engine for Node and the browser*. URL: https://github.com/foliojs/fontkit (elérés dátuma 2026. 11. 08.).

[21] Fontsource. *Fontsource – Self-host Open Source fonts*. URL: https://fontsource.org/ (elérés dátuma 2026. 11. 08.).

[22] Erich Gamma és tsai. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[23] Michael Goldapp. „Approximation of circular arcs by cubic polynomials". *Computer Aided Geometric Design* 8.3 (1991), 227–238. old. DOI: 10.1016/0167-8396(91)90007-X.

[24] Google. *CSS API update*. URL: https://developers.google.com/fonts/docs/css2 (elérés dátuma 2026. 11. 08.).

[25] Graphite. *Graphite: free, open source vector graphics editor*. URL: https://graphite.art/ (elérés dátuma 2026. 11. 12.).

[26] Inkscape. *About Inkscape*. URL: https://inkscape.org/about/ (elérés dátuma 2026. 11. 08.).

[27] Inkscape. *Inkscape Beginners' Guide: Boolean Operations*. URL: https://inkscape-manuals.readthedocs.io/en/1.1/boolean-operations.html (elérés dátuma 2026. 11. 11.).

[28] Inkscape. *Release notes/1.3*. URL: https://wiki.inkscape.org/wiki/Release_notes/1.3 (elérés dátuma 2026. 11. 11.).

[29] Jakob Nielsen. *10 Usability Heuristics for User Interface Design*. URL: https://www.nngroup.com/articles/ten-usability-heuristics/ (elérés dátuma 2026. 11. 08.).

[30] Angus Johnson. *Clipper – an open source freeware polygon clipping library*. URL: https://angusj.com/delphi/clipper/documentation/Docs/Overview/_Body.htm (elérés dátuma 2026. 11. 08.).

[31] jsdom. *jsdom: A JavaScript implementation of various web standards*. URL: https://github.com/jsdom/jsdom (elérés dátuma 2026. 11. 08.).

[32] Jürg Lehni és Jonathan Puckey. *Paper.js – PathItem*. URL: https://paperjs.org/reference/pathitem (elérés dátuma 2026. 11. 08.).

[33] Francisco Martínez, Antonio Jesús Rueda és Francisco Ramón Feito. „A new algorithm for computing Boolean operations on polygons”. *Computers & Geosciences* 35.6 (2009), 1177–1185. old. DOI: 10.1016/j.cageo.2008.08.009.

[34] MDN Web Docs. *Bézier curve*. URL: https://developer.mozilla.org/en-US/docs/Glossary/Bezier_curve (elérés dátuma 2026. 11. 06.).

[35] MDN Web Docs. *<canvas>: The Graphics Canvas element*. URL: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/canvas (elérés dátuma 2026. 11. 12.).

[36] MDN Web Docs. *Clipboard API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Clipboard_API (elérés dátuma 2026. 11. 08.).

[37] MDN Web Docs. *FontFace*. URL: https://developer.mozilla.org/en-US/docs/Web/API/FontFace (elérés dátuma 2026. 11. 08.).

[38] MDN Web Docs. *Local Font Access API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Local_Font_Access_API (elérés dátuma 2026. 11. 08.).

[39] MDN Web Docs. *pointer-events*. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/pointer-events (elérés dátuma 2026. 11. 08.).

[40] MDN Web Docs. *stroke-linejoin*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Attribute/stroke-linejoin (elérés dátuma 2026. 11. 02.).

[41] MDN Web Docs. *SVG animation with SMIL*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Guides/SVG_animation_with_SMIL (elérés dátuma 2026. 11. 11.).

[42] MDN Web Docs. *SVG attribute reference*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Attribute (elérés dátuma 2026. 11. 02.).

[43] MDN Web Docs. *SVG element reference*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element (elérés dátuma 2026. 11. 02.).

[44] MDN Web Docs. *<clipPath>*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/clipPath (elérés dátuma 2026. 11. 11.).

[45] MDN Web Docs. *<defs>*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/defs (elérés dátuma 2026. 11. 08.).

[46] MDN Web Docs. *<filter>*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/filter (elérés dátuma 2026. 11. 11.).

[47] MDN Web Docs. *<g>*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/g (elérés dátuma 2026. 11. 08.).

[48] MDN Web Docs. *<mask>*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/mask (elérés dátuma 2026. 11. 11.).

[49] MDN Web Docs. *touch-action*. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/touch-action (elérés dátuma 2026. 11. 08.).

[50] MDN Web Docs. *transform*. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Attribute/transform (elérés dátuma 2026. 11. 11.).

[51] MDN Web Docs. *TypeScript*. URL: https://developer.mozilla.org/en-US/docs/Glossary/TypeScript (elérés dátuma 2026. 11. 02.).

[52] MDN Web Docs. *Using media queries*. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries (elérés dátuma 2026. 11. 08.).

[53] MDN Web Docs. *Web Storage API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API (elérés dátuma 2026. 11. 08.).

[54] Mediamodifier. *Free SVG Editor*. URL: https://mediamodifier.com/free-svg-editor (elérés dátuma 2026. 11. 11.).

[55] Microsoft. *Playwright – Installation*. URL: https://playwright.dev/docs/intro (elérés dátuma 2026. 11. 08.).

[56] Mike Kamermans (Pomax). *A Primer on Bézier Curves*. URL: https://pomax.github.io/bezierinfo/ (elérés dátuma 2026. 11. 08.).

[57] Barton P. Miller, Lars Fredriksen és Bryan So. „An empirical study of the reliability of UNIX utilities". *Communications of the ACM* 33.12 (1990), 32–44. old. DOI: 10.1145/96267.96279.

[58] Chris Okasaki. *Purely Functional Data Structures*. Cambridge: Cambridge University Press, 1998.

[59] Pekka Parhi, Amy K. Karlson és Benjamin B. Bederson. „Target size study for one-handed thumb use on small touchscreen devices". *Proceedings of the 8th Conference on Human-Computer Interaction with Mobile Devices and Services (MobileHCI '06)*. ACM, 2006, 203–210. old. DOI: 10.1145/1152215.1152260.

[60] Penpot. *Penpot: The free and open source online design tool*. URL: https://penpot.app/design/design-tool (elérés dátuma 2026. 11. 12.).

[61] Urs Ramer. „An iterative procedure for the polygonal approximation of plane curves". *Computer Graphics and Image Processing* 1.3 (1972), 244–256. old.

[62] SIL International. *SIL Open Font License*. URL: https://openfontlicense.org/ (elérés dátuma 2026. 11. 08.).

[63] Ivan Edward Sutherland. *Sketchpad: A man-machine graphical communication system*. Techn. jel. UCAM-CL-TR-574. Az 1963-as MIT doktori értekezés újrakiadása. University of Cambridge, Computer Laboratory, 2003. DOI: 10.48456/tr-574.

[64] Svelte. *Overview*. URL: https://svelte.dev/docs/svelte/overview (elérés dátuma 2026. 11. 08.).

[65] Svelte. *Stores*. URL: https://svelte.dev/docs/svelte/stores (elérés dátuma 2026. 11. 08.).

[66] SVG-Edit. *SVG-Edit forráskódja, 7.4.2-es változat*. URL: https://github.com/SVG-Edit/svgedit (elérés dátuma 2026. 11. 11.).

[67] SVG-Edit. *SVG-Edit online demó*. URL: https://svgedit.netlify.app/index.html (elérés dátuma 2026. 11. 08.).

[68] Bala R. Vatti. „A generic solution to polygon clipping". *Communications of the ACM* 35.7 (1992), 56–63. old. DOI: 10.1145/129902.129906.

[69] Vite. *Why Vite*. URL: https://vite.dev/guide/why (elérés dátuma 2026. 11. 08.).

[70] Vitest. *Test Environment*. URL: https://vitest.dev/guide/environment (elérés dátuma 2026. 11. 08.).

[71] W3C. *Pointer Events*. URL: https://www.w3.org/TR/pointerevents3/ (elérés dátuma 2026. 11. 08.).

[72] W3C. *Scalable Vector Graphics (SVG) 1.0 Specification, W3C Recommendation, 2001. szeptember 4.* URL: https://www.w3.org/TR/2001/REC-SVG-20010904/ (elérés dátuma 2026. 10. 26.).

[73] W3C. *Scalable Vector Graphics (SVG) 2*. URL: https://www.w3.org/TR/SVG2/ (elérés dátuma 2026. 10. 26.).

[74] W3C. *SVG 2 – Implementation Notes*. URL: https://www.w3.org/TR/SVG2/implnote.html (elérés dátuma 2026. 11. 03.).

[75] W3C. *SVG 2 – Paint Servers: Solid Colors, Gradients, Patterns and Hatches*. URL: https://www.w3.org/TR/SVG2/pservers.html (elérés dátuma 2026. 11. 10.).

[76] W3C. *SVG 2 – Painting: Filling, Stroking and Marker Symbols*. URL: https://www.w3.org/TR/SVG2/painting.html (elérés dátuma 2026. 10. 27.).

[77] W3C. *Understanding Success Criterion 2.5.8: Target Size (Minimum)*. URL: https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html (elérés dátuma 2026. 11. 08.).

[78] W3C. *WOFF File Format 2.0*. URL: https://www.w3.org/TR/WOFF2/ (elérés dátuma 2026. 11. 08.).

[79] W3Techs. *Usage statistics of SVG for websites*. URL: https://w3techs.com/technologies/details/im-svg (elérés dátuma 2026. 11. 11.).

[80] WHATWG. *HTML Living Standard – Web storage*. URL: https://html.spec.whatwg.org/multipage/webstorage.html (elérés dátuma 2026. 11. 08.).

A mellékelt adathordozó tartalma

Az adathordozó a következőket tartalmazza:

- `dolgozat.pdf`: A dolgozat PDF formátumban.
- `hasznalati.pdf`: A használati útmutató PDF formátumban.
- `dolgozat/`: A dolgozat LaTeX forráskódja. Fordításához szükségesek a `pdflatex` és `bibtex` programok. Az `svg` csomag dependenciájaként az `Inkscape` program is szükséges.
- `program/`: A szerkesztő teljes forráskódja a tesztekkel együtt. Az `src` mappa az alkalmazás kódját, a `tests` mappa az automatikus teszteket, a `docs` mappa az architektúra leírását, a tervezési döntéseket és egy angol nyelvű felhasználói útmutatót tartalmaz. A `dist` mappában az alkalmazás lefordított, éles változata, az `eval` mappában pedig a 6.2. szakasz mérőszkriptjei és a mérésekhez felhasznált fájlok listája található.

A szerkesztő futtatása telepítés nélkül

A `program/dist` mappa tartalmát bármely statikus webszerver kiszolgálhatja. Ha a gépen van Python 3, a mappában kiadott `python3 -m http.server 8000` parancs után a szerkesztő a `http://localhost:8000` címen nyitható meg. Az `index.html` fájl közvetlen megnyitása nem elég, mert a böngészők a `file://` címről betöltött JavaScript-modulokat biztonsági okból általában letiltják.

Fordítás a forráskódból

A fordításhoz Node.js 20.19 vagy 22.12, illetve ezeknél újabb változat szükséges, ezt a Vite 7 írja elő. A `program` mappában kiadandó parancsok:

`npm install` telepíti a függőségeket, ehhez internetkapcsolat szükséges;

`npm run dev` elindítja a fejlesztői szervert, a szerkesztő a kiírt címen, alapértelmezés szerint a `http://localhost:5173` címen nyitható meg böngészőben;

`npm run build` újra elkészíti az éles változatot a `dist` mappába;

`npm run preview` helyben kiszolgálja az elkészült éles változatot;

`npm test` lefuttatja az automatikus teszteket.

A szerkesztő egy Chromium-alapú böngészőben, például Google Chrome vagy Microsoft Edge alatt használható a legteljesebb funkcionalitással, mert a helyi betűtípusok elérése csak ezekben támogatott. A Google Fonts-betűtípusok letöltéséhez internetkapcsolat szükséges, a szerkesztő többi része hálózat nélkül is működik.